The Chinese University of Hong Kong, Shenzhen
Personal study materials · Unofficial

CSC5041 数据库系统导论

中文复习笔记

Weeks 1–5

Adapted from the supplied academic report template.

Original teaching materials remain unchanged.

LaTeX edition: 20 October 2026.

2026-10-20

目录

01 数据库基础：中文学习笔记 **5**

阅读说明与学习目标 . . . 5
1. 四个基础概念 . . . 5
2. 为什么不直接用文件管理所有数据? . . . 5
3. DBMS 提供什么? . . . 6
4. 数据模型、元数据、模式与实例 . . . 7
5. 数据库设计的三个阶段 . . . 8
6. 数据库语言与用户 . . . 8
7. 三级模式架构 Three-schema architecture【补充】 . . . 9
8. 数据独立性 Data independence【补充】 . . . 9
9. 读懂 DBMS 组件图 . . . 10
10. 数据库类型：掌握分类，不背实时排名 . . . 11
11. 易混点速查 . . . 11
12. 复习清单与英文表达 . . . 11
来源与整理记录 . . . 12

02 ER 概念建模：中文学习笔记 **13**

阅读说明与学习目标 . . . 13
1. ER 模型解决什么问题? . . . 13
2. 实体、实体类型与实体集 . . . 13
3. 属性类型：先区分三个维度 . . . 14
4. 键：唯一性和最小性要分开检查 . . . 14
5. 强实体、弱实体与部分键 . . . 15
6. 关系、关系属性与角色 . . . 16
7. 基数约束：问两边的“最多” . . . 17
8. 参与约束：再问每边的“至少” . . . 17
9. 课堂 ER 符号速查 . . . 18
10. 贯穿例子：从图书馆需求到 ER 图 . . . 18
11. 从文字建模的操作顺序 . . . 20
12. 易混点速查 . . . 20
13. 复习清单与英文表达 . . . 20
来源与整理记录 . . . 21

03 关系模型与完整性约束：中文学习笔记 **22**

阅读说明与学习目标 . . . 22
1. 从 ER 模型走到关系模型 . . . 22
2. 模式、实例、度与基数 . . . 22
3. 一个关系的基本性质 . . . 23
4. 超键、候选键与主键 . . . 24
5. 外键：方向、重复值与复合引用 . . . 25
6. NULL 与外键的边界 . . . 25
7. 三类完整性约束检查什么？ . . . 26
8. 贯穿例子：先明确模式、约束与初始状态 . . . 26
9. 插入：检查新元组与被引用目标 . . . 27
10. 删除：沿外键反向查找引用者 . . . 28
11. 修改：同时考虑旧值与新值 . . . 28
12. 易混点速查 . . . 29
13. 复习清单与英文表达 . . . 29
来源与整理记录 . . . 30

04 ER 到关系模式的映射：中文学习笔记 31
阅读说明与学习目标 . . . 31
1. 映射的输入、输出与记号 . . . 31
2. 七步映射总览 . . . 32
3. Step 1：强实体与普通属性 . . . 32
4. Step 2：弱实体与标识关系 . . . 33
5. Step 3：二元 1:1 关系 . . . 34
6. Step 4：二元 1:N 关系 . . . 34
7. Step 5：二元 M:N 关系 . . . 35
8. Step 6：多值属性 . . . 35
9. Step 7：n 元关系与主键判断 . . . 36
10. 贯穿例子：把课件公司 ER 图整理成模式 . . . 37
11. 映射后仍要检查的约束 . . . 38
12. 一套可执行的答题顺序 . . . 38
13. 易混点速查 . . . 39
14. 复习清单与英文表达 . . . 39
来源与整理记录 . . . 40

05 关系代数：中文学习笔记 41
阅读说明与学习目标 . . . 41
1. 关系代数的记号与闭包 . . . 41

2. 贯穿例子：先写清输入 . . . 41
3. 选择 σ：筛选元组，保留全部属性 . . . 43
4. 投影 π：保留指定属性并消除重复 . . . 43
5. 重命名 ρ：处理角色和同名列 . . . 44
6. 并、交、差：先保证模式相容 . . . 44
7. 积与连接：从所有配对到正确配对 . . . 45
8. 组合查询：保留身份，最后输出姓名 . . . 45
9. 除法 $\div$：表达“每一个目标都满足” . . . 46
10. 除法的差集展开与空目标 . . . 47
11. γ：分组与聚合 . . . 47
12. 等价改写：先检查前提 . . . 48
13. 从英文需求写表达式的步骤 . . . 48
14. 易混点、复习清单与英文表达 . . . 49
来源与整理记录 . . . 50

06 SQL 查询：中文学习笔记 51
阅读说明与学习目标 . . . 51
1. 六个常用子句：书写顺序与逻辑顺序 . . . 51
2. 贯穿例子：在 05 的基础上加入学期与缺失成绩 . . . 52
3. 基本查询、别名、字符串与 DISTINCT . . . 53
4. NULL 与三值逻辑 . . . 54
5. 多表连接：先确定完整配对条件 . . . 54
6. 自连接与排序 . . . 55
7. 聚合：COUNT、SUM、AVG 与 NULL . . . 55
8. GROUP BY 与 HAVING：每组一行 . . . 56
9. 子查询：先区分标量与多行结果 . . . 57
10. EXISTS、NOT EXISTS 与相关子查询 . . . 57
11. NOT IN 遇到 NULL：反例要会解释 . . . 58
12. UNION、INTERSECT、EXCEPT 与身份粒度 . . . 58
13. “每个目标都满足”：没有缺失目标 . . . 59
14. 课件其余内容与本轮范围 . . . 60
15. 写查询的检查顺序与复习清单 . . . 60
来源与整理记录 . . . 61

01 数据库基础：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

阅读说明与学习目标

本篇对应数据库基础，不提前展开 ER 作图、关系代数或 SQL 查询技巧。主要依据教师导论课件第 17–57 页，页码按 PPT 幻灯片顺序计数。第 7、8 节是教学计划第 1 页明确列出、但导论 PPT 未充分展开的补充讲解，不能当作教师课件原文。文中的例子用于解释概念，并非真实考试题或实测结果。

学完后，应能区分数据、数据库、DBMS 和数据库系统；解释文件方式的局限；区分模式与实例；说明设计层次、三级模式和数据独立性；按功能读懂 DBMS 组件图。

1. 四个基础概念

来源：导论第 17–18、33、41、45 页。

概念	定义	学生选课系统中的例子
数据 Data	可以记录、具有隐含意义的已知事实	学号 S01、姓名 Amy、课程 CSC5041
数据库 Database	相互关联、描述现实世界某一部分的数据集合	学生、课程、选课记录的集合
数据库管理系统 DBMS	支持创建、维护、查询和管理数据库的软件	PostgreSQL 等数据库管理软件
数据库系统 Database system	按课件的简化定义，数据库与 DBMS 的组合	选课数据及管理这些数据的 DBMS

数据库保存数据，DBMS 管理数据。 SQL 是与关系数据库交互的语言，不是数据库或 DBMS。应用程序通过 DBMS 读写数据；例如选课网页属于应用程序，不等同于 DBMS。

“相互关联” 不要求每条记录直接连接所有其他记录，而是指数据共同描述一个有意义的应用领域。

结构化与非结构化数据

来源：第 34 页。

- **结构化数据 Structured data**：具有预先规定的组织方式，例如每条学生记录包含学号、姓名、专业。
- **非结构化数据 Unstructured data**：没有按固定记录模式组织，例如自由文本、录音、图片；分析前通常需要额外处理。

“非结构化” 不是 “完全没有信息或内部结构”；“放在电脑文件里” 也不代表它就是关系表。

2. 为什么不直接用文件管理所有数据？

来源：第 35–40 页。

课件的选课例子使用三个文件：

```
STUDENT(name, id, major)
COURSE(code, name, department)
SELECTION(student_id, course_code, term)
```

想判断 Amy 是否选了一门课，需要先从学生文件得到她的学号，再到选课文件匹配学号、课程和学期。文件本身可以保存这些信息，困难在于每个应用都要反复实现访问、约束、并发和恢复逻辑。

文件方式的问题	含义	说明例子
数据冗余 Redundancy	同一事实被重复保存	多个部门各存一份学生地址
数据不一致 Inconsistency	同一事实的副本互相冲突	地址只在一个文件中更新
访问困难	新查询需要新的程序或处理逻辑	每增加一个统计需求就重新编写文件处理代码
数据分散/隔离 Data isolation	数据分散在不同文件和格式中，难以统一访问	学生资料是 CSV，课程资料是另一种格式
完整性问题 Integrity problems	业务约束散落在程序里，难以统一维护	一段程序检查余额非负，另一段忘记检查
原子更新困难	多步更新可能只完成一部分	转账扣款成功、入账前系统崩溃
并发访问问题	多个用户的操作互相干扰	两次取款读到相同旧余额，后写覆盖先写

冗余可能导致不一致，但重复存储并不意味着此刻一定存在冲突。课件强调消除不必要的冗余；实践中有目的、受控制的复制与任意重复不同。

这里的 **data isolation 指文件和格式造成的数据分散**，不要与后续事务章节的 isolation 混淆。

两个必须能讲清的例子

转账与原子性 Atomicity：从 A 转出 100 元、向 B 转入 100 元构成一个整体。要么两步都生效，要么失败后撤销已做的部分，不能只扣不加。

并发与丢失更新 Lost update（术语补充）：初始余额 100。两个操作都读取 100，各取 50，随后都写入 50。若两次取款均被接受，正确余额应为 0，最终却为 50。问题来自操作交错和旧值覆盖，不能仅靠一次 “余额是否足够” 的检查解决。

本篇只理解问题；并发控制和恢复算法留待后续课程。

3. DBMS 提供什么？

来源：第 41–42、44、47–48 页。

DBMS 支持定义数据结构、构建和装载数据库、查询数据、插入/删除/修改数据，以及管理访问。

能力	需要理解的作用
数据独立性 Data independence	降低应用对底层数据表示和存储的依赖
高效访问 Efficient access	由数据库机制支持数据查询和更新，而非让每个应用手写文件扫描
完整性 Integrity	通过已声明的约束维持有效数据状态

能力	需要理解的作用
安全性 Security	控制谁可以读取或修改哪些数据
数据管理 Administration	集中管理结构、权限及运行维护
并发与恢复 Concurrency and recovery	协调共享访问，并在故障后恢复到一致状态
减少开发工作	让应用复用共同的数据管理能力

完整性与安全性不同：阻止不合法的选课记录是完整性；阻止学生修改他人成绩是安全性。DBMS 提供机制，但业务规则仍需正确设计与配置。

课件还提出及时响应、可修改、健壮、多用户、安全等要求。这些是系统目标，不表示使用数据库就会自动满足所有性能或正确性要求。

4. 数据模型、元数据、模式与实例

来源：第 50、53 页；元数据说明结合第 56 页结构图。

数据模型 Data model

数据模型是一套描述数据库结构的概念和规则。它回答 “可以用什么结构表达数据”，而不是某个时刻具体存了哪些记录。

层次	关注点	例子
高层/概念模型 Conceptual model	应用中的实体、属性和关系	ER 模型
实现/记录模型 Implementation or record-based model	可用于数据库逻辑实现的数据结构	关系、层次、网状模型
低层/物理模型 Physical model	记录格式、存储组织、访问路径	记录如何排列、如何定位

“关系模型” 是一类模型；“学生表包含哪些列” 是某个数据库采用该模型得到的模式；“表里当前有哪些学生” 是实例。

模式 Schema 与实例 Instance / state

```
模式：Student(sid, name, major)

时刻 t1 的实例：
(S01, Amy, CS)
(S02, Ben, Math)

时刻 t2 的实例：
(S01, Amy, CS)
(S02, Ben, Math)
(S03, Chen, CS)
```

- 插入第三名学生：**实例变化，模式不变**。
- 将 Amy 的专业改成 Math：**实例变化，模式不变**。
- 给 Student 增加 email 属性：**模式变化**。
- 表已经定义但没有记录：存在模式，对应一个空实例。

元数据 Metadata 是描述数据的数据，如表名、列名、类型和约束；系统目录/数据字典保存这类描述，供 DBMS 使用。学生姓名 Amy 是业务数据，不是元数据。

5. 数据库设计的三个阶段

来源：第 54 页。

阶段	主要问题	选课例子
概念设计 Conceptual design	应用需要表达什么事物和业务规则?	有学生、课程以及选课关系
逻辑设计 Logical design	如何用目标数据模型表示?	转换为 Student、Course、Selection 等关系及约束
物理设计 Physical design	如何实际存储并高效访问?	选择文件组织、索引和存储安排

不要在需求尚未弄清时直接从磁盘格式开始设计。概念设计也不是 “开始填写具体记录”。

这三个设计阶段与第 7 节的三级模式有关，但不是同一个分类。设计阶段描述工作过程；三级模式描述数据抽象和接口层次。尤其不要把 “三个设计阶段” 当作 “三模式” 的标准答案。

6. 数据库语言与用户

来源：第 49、55 页。

DDL 与 DML

类别	用途	识别用的示例，SQL 语法后续再学
DDL：Data Definition Language	定义或修改数据库结构和约束	CREATE TABLE、ALTER TABLE、DROP TABLE
DML：Data Manipulation Language	查询、插入、删除、修改数据	SELECT、INSERT、DELETE、UPDATE

本课导论采用广义 DML，包括查询；不要因为其他资料把 SELECT 单列为 DQL 就认为课件分类错误。

声明式/非过程式 Declarative / non-procedural：说明想要什么结果，让 DBMS 决定如何执行。例如 “返回 CS 专业学生的姓名”。

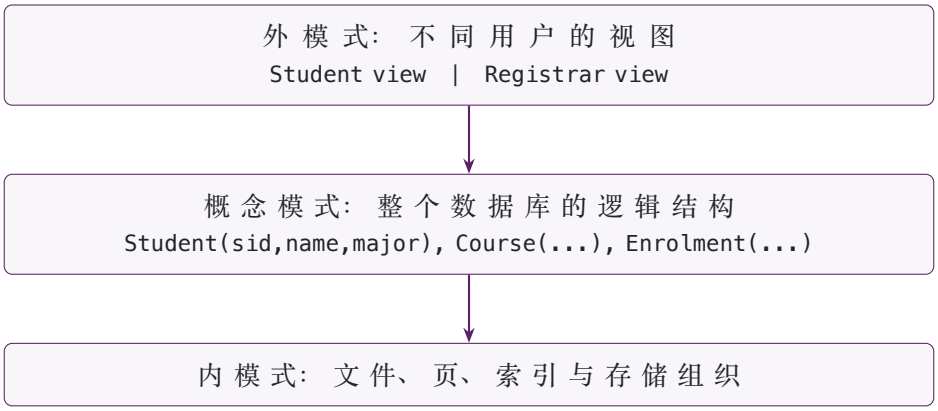
过程式 Procedural：给出完成任务的操作过程。例如逐条读取学生记录、检查专业、输出姓名。SQL 查询通常是声明式的，但课程后面会学习 PL/pgSQL 的变量、分支和循环。

谁使用数据库?

用户	职责
DBA：Database Administrator	管理模式、安全与授权、可用性、故障恢复和性能调优
应用程序员 Application programmer	根据应用需求编写访问数据库的程序
最终用户 End user	使用查询工具或应用程序完成业务任务

DBA 与应用程序员是职责不同，不意味着现实组织中绝对不能由同一人承担。

7. 三级模式架构 Three-schema architecture【补充】



逻辑独立性：改变概念模式，保持外模式及应用不变。
物理独立性：改变内模式，保持概念模式不变。

范围依据：教学计划第 1 页 Week 1。概念定义参考下方补充来源；不是 PPT 的逐页摘录。

层次	描述什么
外模式 External schema	某类用户或应用所见的数据视图，可有多个
概念模式 Conceptual schema	数据库整体的逻辑结构，包含数据及约束
内模式 Internal schema	数据的物理存储组织和访问方式

上下层通过映射联系；外模式可以隐藏数据，也可以提供计算后的表示。各用户视图并不意味着各自保存一份独立数据库。

本节核心定义参考 UCT 数据库课程第 1 章 的 “ANSI/SPARC three-level architecture”。

贯穿例子（本笔记自编）：教务员需要完整选课资料，学生只需要自己的选课清单，这是不同外模式。Student、Course、Selection 及其约束构成整体逻辑结构。底层的文件组织和访问路径属于内模式。

注意：这里的三级模式不是应用部署中的 “界面层、业务层、数据库层”。后者是另一种架构分类。

8. 数据独立性 Data independence【补充】

范围依据：教学计划第 1 页及导论第 42 页。

类型	允许哪一层改变?	希望哪一层保持不变?
物理数据独立性 Physical data independence	内模式	概念模式及其上层应用接口
逻辑数据独立性 Logical data independence	概念模式	外模式及使用它们的应用

映射需要随下层变化而调整。数据独立性不表示所有结构变化都能自动兼容。

定义参考 Emory 的三级模式课程材料；物理层与逻辑层分离亦见 Oracle Database Concepts。

如何判断?

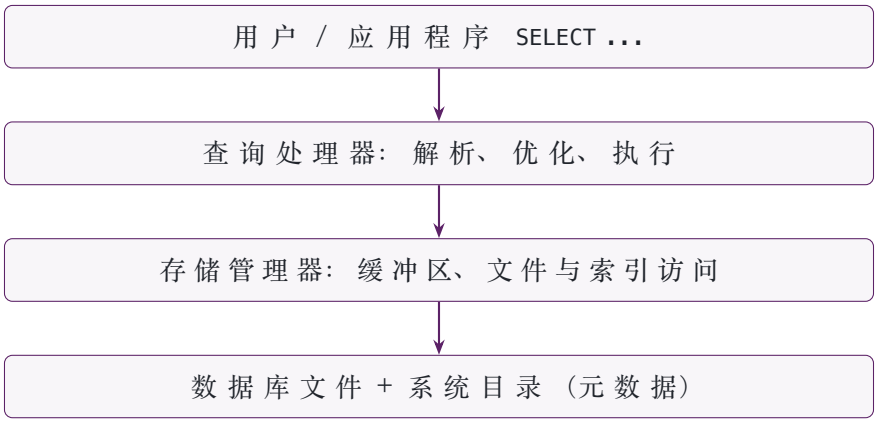
例 A（自编）：调整底层文件组织，Student 的逻辑定义和应用查询保持不变。改变的是物理存储，因此体现物理数据独立性。

例 B（自编）：原来的学生逻辑结构拆成基本信息和联系信息两部分，通过更新视图映射，旧应用仍看到原先的学生信息接口。改变的是整体逻辑结构，因此体现逻辑数据独立性。前提是旧接口需要的信息仍能得到，不能假定任意拆表都无影响。

例 C（自编）：给 Amy 修改电话号码只是数据内容变化，不是逻辑数据独立性的例子。

记忆顺序：**用户视图** → **整体逻辑结构** → **物理存储**。先判断变化在哪层，再判断上层是否被隔离。

9. 读懂 DBMS 组件图



简化的概念流程；权限、完整性、并发与恢复机制贯穿请求处理。

来源：导论第 56 页，已查看原图。以下为功能性概括，不是对真实产品固定执行顺序的保证。

图中组件	基本职责
DDL compiler	处理结构定义，将相关描述提供给系统目录
System catalog / data dictionary	保存数据库结构等元数据
Query compiler	处理查询，使其成为可供后续执行的表示
Query optimizer	选择合适的查询执行方案
Runtime database processor	在运行时处理数据库请求
Stored data manager	管理底层数据访问和 I/O
Concurrency control / backup / recovery subsystems	协调并发，支持备份与故障恢复

对交互式查询，可用下面的简化流程理解：

```
用户查询 → 查询编译 → 优化 → 运行时处理 → 数据访问 → 返回结果
                        ↕
                  系统目录提供元数据
```

原图还画出了应用程序的预编译、DML 编译和宿主语言编译路径。理解它们是不同入口即可，不需要在本篇展开编译器实现。

两个视角分开记：三级模式讨论“数据怎样分层描述”；组件图讨论“哪些软件模块处理请求”。

10. 数据库类型：掌握分类，不背实时排名

来源：第 20、51–52 页。

关系数据库以关系/表为主要组织方式；课件还列出键值、文档、图和列族等类型。课件中的例子包括 PostgreSQL、MySQL（关系），Redis（键值），MongoDB（文档），Cassandra（列族）。这些用于识别模型，不构成最新市场排名或完整产品能力描述。

NoSQL 不等于“没有结构”，关系数据库也不只是一堆互不关联的电子表格。当前测验的后续复习以关系数据库为主。

11. 易混点速查

容易混淆	正确区分
Database / DBMS	数据集合 / 管理该集合的软件
Schema / instance	结构描述 / 某时刻实际内容
Data model / schema	描述结构的概念体系 / 用它定义的具体数据库结构
Metadata / data	对结构等信息的描述 / 应用领域中的事实
Redundancy / inconsistency	重复存储 / 副本发生矛盾
Integrity / security	数据是否符合规则 / 谁有权访问和操作
DDL / DML	定义结构 / 查询和修改内容
设计阶段 / 三级模式	设计工作过程 / 数据抽象层次
Physical / logical independence	存储变化隔离 / 整体逻辑结构变化隔离
原子性 / 并发	单个工作整体是否完整生效 / 多个操作如何交错共享数据

12. 复习清单与英文表达

以下是自我复习目标，不是正式模拟测试，也不代表官方分值。

- ☐ 不看笔记区分四个基础概念，并各给一个例子。
- ☐ 用因果关系解释文件方式的至少四个问题。
- ☐ 讲清转账失败和并发取款分别说明什么问题。
- ☐ 判断插入记录、修改属性值、增加列分别改变模式还是实例。
- ☐ 区分数据模型、设计阶段和三级模式。
- ☐ 对给定变化指出其层次，并解释物理或逻辑数据独立性。
- ☐ 用功能说明 DDL、DML、DBA 及核心 DBMS 组件。

可用于英文简答的表达：

> A database is a collection of related data, whereas a DBMS is the software used to create and manage it.

> A schema describes the database structure; an instance contains the actual data at a particular time.

Physical data independence allows storage changes without changing the conceptual schema.

Logical data independence allows conceptual-schema changes while preserving existing external interfaces through suitable mappings.

Atomicity means that a transaction's changes take effect as a whole or do not take effect at all.

来源与整理记录

- 教师导论：Week 1A - IntroductionNEW.pptx，主要第 17–57 页；组件图为第 56 页。
- 教师计划：CSC5041_Teaching_Plan.pdf，第 1 页 Week 1 与 Test 1。
- 补充网页仅用于第 7–8 节的架构与独立性定义；自编例子已明确标注。补充资料不决定教师实际考查深度。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入版本：导论 SHA-256 `d36c2d15a77083931046168738d794558c2e9821d452701f64ef524e45e87a7c`；教学计划 SHA-256 `b62c7356961bf10585f2e7d5fbf29bcb8401970a99d09d80fe6675fa63cea463`。

02 ER 概念建模：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

阅读说明与学习目标

本篇接续 01 数据库基础，对应教学计划第 1 页 Week 2，以及 Tutorial 1 第 1 页 Question 1(a)–(b)。主要依据教师 Week 1B - Conceptual DB Design.pptx，下文 “第 x 页” 均指该文件的第 x 张幻灯片。

学完后，应能从文字需求识别实体、属性、键与关系，判断弱实体，区分基数和参与约束，并用课堂符号画出 ER 图。关系模型的完整性约束留到 03，系统的 ER-to-relational mapping 留到 04。本篇中的解释性补充与自编例子不代表新增考试要求。

课件第 70 页明确要求：**“You must use the notation in the lecture for this course.”** 本篇使用课件的 Chen 风格符号，不用乌鸦脚符号替代。

1. ER 模型解决什么问题?

来源：第 2–10、54–56、69–70 页。

ER（Entity-Relationship）模型把应用需求组织为**实体、属性、关系及约束**，属于概念设计。

```
应用需求 → 识别事物及其联系 → ER 图与约束说明
                                  ↓ 后续 04
                              关系模式及主外键
```

例如，“每本书必须由一个出版社出版” 包含两个实体类型 Book、Publisher，一个关系 PUBLISH，以及 “每本书恰好一个出版社” 的约束。只画两个框再连线，会遗漏业务规则。

概念设计首先回答 “需要表达什么”。本篇不从 SQL 建表语句或索引开始，也不把所有外键直接画成 ER 属性。

2. 实体、实体类型与实体集

来源：第 10–14、24–27 页。

概念	含义	例子
实体 Entity	可以被区分的具体对象或概念对象	学号为 S01 的学生 Amy
实体类型 Entity type	定义一类实体共同的属性和结构	Student，有 sid、name 等属性
实体集 Entity set	某时刻该类型的实体集合	当前登记的全部学生
属性 Attribute	描述实体或关系的性质	学生姓名、选课成绩
域 Domain / value set	属性允许取得的值的集合	某属性允许的日期值

实体类型对应结构，实体集对应当前内容，可联系 01 的 schema / instance 理解。实体不必是物体，课程、账户、预约也可以建模为实体。

怎样判断实体还是属性? 看需求是否需要独立识别它、记录它自己的性质，并让它参与关系。例如只记录出版社名称时可以考虑一个属性；若要记录出版社编号、名称及其出版的多本书，Publisher 更适合成为实体类型。不能仅凭 “它是名词” 就决定画实体框。

3. 属性类型：先区分三个维度

来源：第 14–23、27–28 页。

维度	类型	含义与例子
能否按业务含义拆分	简单/原子 Simple / atomic	当前设计不再拆分，如 salary
能否按业务含义拆分	复合 Composite	可拆为有独立含义的部分，如 address(city, street, postcode)
每个实体同一时刻有几个值	单值 Single-valued	每个学生只有一个 dateOfBirth
每个实体同一时刻有几个值	多值 Multivalued	一个学生可有多个 phoneNumber
值从哪里来	存储 Stored	直接保存 dateOfBirth
值从哪里来	派生 Derived	根据 dateOfBirth 和参考日期计算 age

这些不是六个互斥类别。例如 address 可以是**单值且复合**；若一个人允许登记多个地址，address 也可以是**多值且复合**。课件第 15 页将多值属性与 simple 对照；学习时应分别判断 “能否拆分” 和 “有几个值”，不能把 simple 与 single-valued 当同义词。

复合属性与多值属性

```
复合：一个地址 = (city, street, postcode)
多值：多个电话号码 = {130..., 131...}
```

复合属性的分支表示**组成部分**，多值属性的双椭圆表示**值的数量可以多于一个**。若应用只把地址作为整体使用，课件第 21 页指出可以不拆分；是否拆分取决于需要支持的操作。

存储与派生

年龄依赖出生日期和参考日期。每年手动修改 age 容易使它与出生日期矛盾，因此本课概念模型把 age 作为派生属性，把 dateOfBirth 作为存储属性。这里讨论概念含义，不规定真实产品必须采用哪种缓存实现。

缺失值 NULL

课件第 28 页用 NULL 表示未知或不适用的属性值。例如身高尚未知、某人没有大学学位。NULL 不等于数字 0，也不等于已知的空字符串。SQL 中怎样比较 NULL 留到 06。

4. 键：唯一性和最小性要分开检查

来源：第 29–32 页。

类型	定义
超键 Superkey	能唯一识别该类型实体的一组属性，允许含多余属性
候选键 Candidate key	最小超键：去掉任何一个属性后，都不再保证唯一
主键 Primary key	设计者从候选键中选定的一个
复合键 Composite key	由多个属性组成的键

最小 minimal 不等于属性数最少 minimum：一个类型可以同时有单属性候选键和双属性候选键，只要每个候选键内部都没有多余属性。

自编例子：假定 Student 中 sid 唯一，email 也唯一。

- `{sid}`、`{email}` 都是候选键；可以选 `{sid}` 为主键。
- `{sid, name}` 是超键，不是候选键，因为 name 多余。
- 即使当前三名学生姓名互不相同，也不能据此断定 `{name}` 是键。键必须在需求允许的所有合法状态下唯一。

复合键中的每个属性不需要单独唯一。例如在“编号只在本地区内唯一”的前提下，`{regionCode, localNumber}` 可共同识别一个对象。不要因为它有多个属性，就误画成多值属性。

如果自然属性不能可靠地区分对象，可以引入人工标识符；但“添加 ID”本身不能保证其他业务约束已表达完整。

5. 强实体、弱实体与部分键



自编示例。房号只在楼内唯一，桌号只在房间内唯一。
Desk 的外键为 $(buildingID, roomNo)$，不能只保留 $roomNo$。

来源：第 33–35 页；结合 Tutorial 1 Question 1(a) 的局部编号要求。

概念	判断要点
强实体 Strong entity	有自身的键，可独立识别
弱实体 Weak entity	缺少自身完整的键，需要所有者的标识和自身部分键一起识别
所有者实体 Owner entity	提供识别弱实体所需的上下文
部分键 Partial key / discriminator	在同一个所有者范围内区分弱实体，不保证跨所有者唯一
标识关系 Identifying relationship	将弱实体与其所有者联系起来的关系

课件第 34 页“partially unique”的措辞应理解为**唯一性只在某个所有者范围内成立**，而不是“只能识别实体的一部分属性”。

一个两层弱实体例子

以下据 Tutorial 1 的编号规则解释，不是整题答案：

```
Owner: ownerID 全局唯一
  └ Pet: petNo 只在该 Owner 内唯一
       └ Treatment: treatmentNo 只在该 Pet 内唯一

Pet 的完整标识       = (ownerID, petNo)
Treatment 的完整标识 = (ownerID, petNo, treatmentNo)
```

因此 Pet 的部分键是 petNo；Treatment 的部分键是 treatmentNo。Treatment 的直接所有者 Pet 本身也是弱实体，识别时需要沿所有者链取得完整上下文。

本例每个 Pet 恰好属于一个 Owner，每个 Treatment 恰好属于一个 Pet；弱实体在自己的标识关系中**全参与**。但不能据此推断所有所有者都必须拥有弱实体：Tutorial 1 要求每个 Owner 至少有一只 Pet，却没有要求每只 Pet 必须接受 Treatment。

课件措辞辨析：“弱实体没有自己的主键” 指缺少**自身独立的完整键**，不表示后续映射出的表不能有主键；它可以使用 “所有者键 + 部分键” 的组合，04 再展开。

不要仅凭依赖关系判弱实体。一个员工必须在部门工作，但若 empID 已能独立识别员工，员工仍可为强实体。关键是识别是否依赖所有者。

6. 关系、关系属性与角色

来源：第 38–42、48–53 页。

关系类型描述实体之间的联系；关系实例是一组具体参与者。例如 ENROLLED_IN(Student, Course) 是关系类型，“S01 选修 C01” 是一个关系实例。

属性应挂在哪里？

第 48 页的 WORKS_ON 例子中，fraction 表示某位研究者投入某个项目的时间比例：

```
(Researcher R1, Project P1) → fraction = 0.4
(Researcher R1, Project P2) → fraction = 0.6
```

fraction 描述**特定研究者与特定项目的组合**，应挂在 WORKS_ON 关系上。只挂在 Researcher 上无法区分项目，只挂在 Project 上无法区分研究者。类似地，借阅开始日期通常描述一次借阅联系。

同一对类型可以有多个关系

Tutorial 1 中 Clinic 与 Veterinarian 既有 WORKS_AT，又有 MANAGES。前者表示工作归属，后者表示管理职责，不能合成一条没有名称的线。

“管理者必须在该诊所工作” 还要求 **MANAGES 中的配对必须也是 WORKS_AT 中的配对**。仅画两个关系并分别标基数，不能完整表达这条跨关系约束，应附文字说明。

角色与关系的度

二元关系有两个参与角色，三元关系有三个。课件第 50 页给出 Researcher、Student、Course 的三元示意；第 53 页让同一 Person 类型分别以 Student 和 Researcher 角色参与。

解释性补充：角色数不一定等于不同实体类型数。例如 EMPLOYEE 通过 supervisor、subordinate 两个角色参与 SUPERVISES，仍是二元自关联。画同类型多次参与的关系时，应标角色名。

课件第 52 页建议初学者尝试二元表示，这是建模检查的启发，**不是任意三元关系都能无损拆成三个二元关系的定理**。拆分前应检查能否恢复原来 “哪三者共同关联” 的信息。本篇重点仍是二元关系。

7. 基数约束：问两边的 “最多”

来源：第 43–45 页。

对于二元关系 A—R—B，分别固定一个 A、一个 B，询问它最多关联多少个对方实体。

A:B 的基数比	一个 A 最多对应多少 B?	一个 B 最多对应多少 A?
1:1	1	1
1:N	多个	1
M:N	多个	多个

自编例子：一个 Department 可以有多名 Employee，每名 Employee 最多归属一个 Department，故 Department:Employee 为 1:N。

课件 Chen 标法是：`Department — 1 — WORKS_FOR — N — Employee`。**Department 框旁的 1 不是 “每个部门最多一个员工”**；它表示相对于一名 Employee，关联的 Department 最多为 1。应先写两句自然语言，再标两端数字，避免反向理解。

第 44–45 页的例子同理：一项 Grant 只支持一个 Project，一个 Project 可由多项 Grant 支持，所以 Project:Grant = 1:N；若按 Grant:Project 的顺序写则为 N:1。

`N`、`M` 表示允许多个，不是要求当前恰好存在 N 个，也不表示必须至少两个。**1:N 不能独自表达 “每个员工一定有部门”。**

8. 参与约束：再问每边的 “至少”

来源：第 46–47 页；符号见第 57 页。

参与约束	对该类型中的每个实体的要求	课堂符号
全参与 Total participation	至少参加一次该关系	实体与关系间双线
部分参与 Partial participation	允许不参加，即最少 0 次	单线

部分参与不是 “必须有一些实体不参加”，而是**允许有实体不参加**。即使当前每个人都参加，也不能据此认定规则要求全参与。

将最小值和最大值合起来更容易检查。以下 `(min,max)` 是辅助阅读记号，写在文字表里；正式图仍使用课堂的数字和单双线。

需求用语	每个所讨论实体的参与次数
zero or one / at most one	(0,1)
exactly one	(1,1)
zero or more	(0,N)
one or more / at least one	(1,N)

例如“每个员工必须且只能在一个部门工作；每个部门可以暂时没有员工”：Employee 是 `(1,1)`，Department 是 `(0,N)`。因此 Department:Employee = 1:N，Employee 全参与、Department 部分参与。

注意记号方向：上表某实体的 `(min,max)` 表示“这个实体能关联多少个对方”，与 Chen 基数数字标在图哪一端不是同一阅读规则，不要机械照抄位置。

9. 课堂 ER 符号速查

来源：第 35–37、45、47、57 页，已查看原图。

要表达什么	符号
普通/强实体类型	单矩形
弱实体类型	双矩形
普通关系类型	单菱形
标识关系	双菱形
普通属性	单椭圆
键属性	属性名实线下划线；复合键需标出所有组成属性
部分键	属性名虚线下划线，见第 35 页的示意
多值属性	双椭圆
复合属性	从整体属性椭圆连到各组成属性椭圆
派生属性	虚线椭圆
全参与 / 部分参与	双线 / 单线
基数比	关系两端标 1、N 或 M

双矩形、双菱形、双椭圆、双线有不同含义。**双线表示全参与，不表示多对多；双椭圆表示多值，不表示复合。**键的下划线也不是派生属性的虚线边框。

10. 贯穿例子：从图书馆需求到 ER 图

来源：第 58–68 页。以下重述课件需求并给出学习用重绘；不是新的模拟题。

第一步：实体与属性

实体类型	键	其他属性
Book	bookID	title、price、availability
Reader	readerID	name、phoneNumber、address(street, suburb)
Publisher	publisherID	name
Author	authorID	name、phoneNumber、dateOfBirth

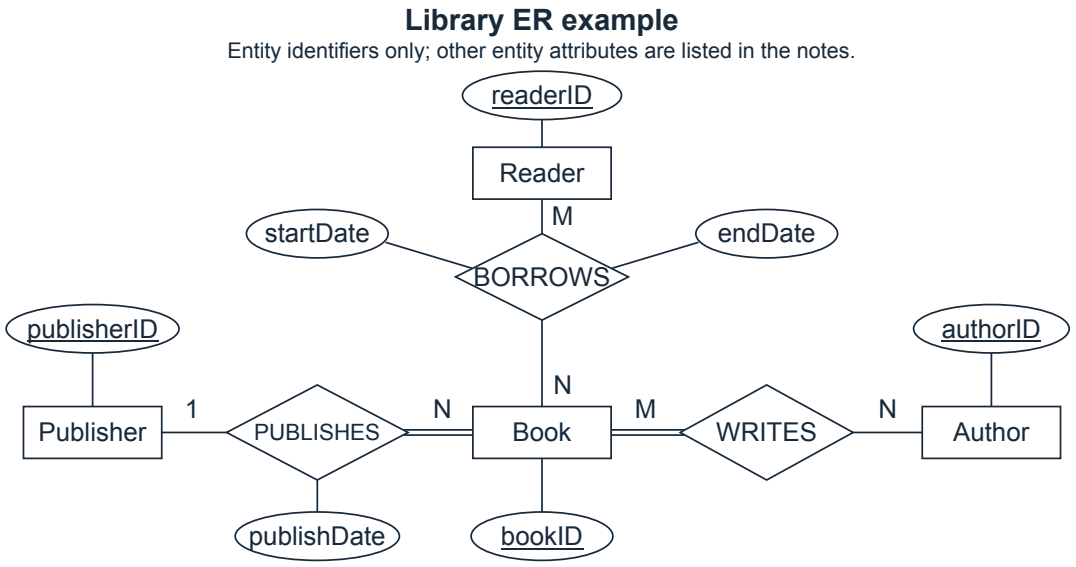


Figure 1. 图书馆 ER 学习图：键、关系属性、基数与参与约束

需求第 58–60 页没有给 Reader 出生日期，第 62 页分步示例却出现 dob。本篇以完整需求和第 68 页总图为准，不把 dob 加入 Reader；Author 的出生日期则有明确要求。此处暴露的是课件局部不一致，不应静默增加需求。

第二步：每条关系都写两个方向

关系	每个左侧实体关联多少右侧实体?	每个右侧实体关联多少左侧实体?	基数与参与	关系属性
Reader—BORROWS—Book	Reader 可借 0 到多本 Book	Book 可被 0 到多个 Reader 借阅	M:N；两边部分参与	startDate、endDate
Publisher—PUBLISHES—Book	Publisher 可出版 0 到多本 Book	Book 必须由恰好 1 个 Publisher 出版	1:N；Publisher 部分、Book 全参与	publishDate
Author—WRITES—Book	Author 可写 0 到多本 Book	Book 必须由 1 到多个 Author 编写	M:N；Author 部分、Book 全参与	无

“一本书可被多个读者借阅”按课件需求理解为数据库记录范围内的联系，不能自行推成同一本实体副本可同时借给多人。

第三步：画图并回读

上图为本笔记重绘，**只显示实体的键属性**；其他实体属性见前表。菱形上的日期属于关系。检查 Book 到 PUBLISHES、WRITES 的双线，再检查 BORROWS 两边的单线，可直接区分三组参与要求。

模型边界（解释性补充）：普通二元关系实例由参与实体配对确定。如果需要保存同一个 Reader 多次借同一个 Book 的独立历史记录，仅在 Reader—Book 关系上挂日期不足以区分重复事件，

应另建 Loan 实体并给出标识规则。课件示意没有完整展开这项历史建模要求；不要把补充当作对原题新增的强制答案。

11. 从文字建模的操作顺序

来源：第 54–55、58–70 页；以下为整理后的解题流程。

1. **读完需求再列实体**：给每个候选实体写一句含义，确认哪些对象需要独立识别。
2. **列属性并分类**：标记复合、多值、派生和可缺失信息，找出各实体的键。
3. **检查弱实体**：编号是否只在某个所有者内部唯一？写出所有者、部分键和完整识别链。
4. **逐条命名关系**：同一对类型可以有多个关系；同类型重复参与时标角色。
5. **分别写两个方向的最小/最大数**：由此确定基数与单双线，不能只写一个 “1:N”。
6. **放置关系属性**：判断属性描述单个对象，还是一组参与者之间的联系。
7. **补充无法由图完整表达的规则**：例如 MANAGES 配对必须属于 WORKS_AT、时间先后、编号顺序；需求没有说明的地方标明假设。
8. **回读整张图**：把框、线、数字再翻译成自然语言，核对是否遗漏或增加了约束。

课件第 55 页的四项质量标准是 correctness、completeness、consistency、readability。不同设计可以合理，但必须准确、完整、一致且可读；不能用 “没有唯一最佳答案” 掩盖漏约束。

12. 易混点速查

容易混淆	正确区分
Entity / entity type / entity set	某个对象 / 一类对象的结构 / 当前这一类对象的集合
Simple / single-valued	能否按含义拆分 / 同一时刻是否只有一个值
Composite attribute / composite key	属性由若干部分组成 / 若干属性共同识别实体
Composite / multivalued	一个值有多个组成部分 / 一个实体可有多个值
Superkey / candidate key	唯一即可 / 唯一且没有多余属性
Partial key / full identifier	所有者范围内唯一 / 连同所有者上下文完整识别
弱实体 / 必须依赖其他实体存在	弱实体的识别依赖所有者；只有存在依赖不足以判弱
Cardinality / participation	最多关联多少个 / 是否必须至少关联一个
Double line / double oval	全参与 / 多值属性
Entity attribute / relationship attribute	描述单个对象 / 描述参与者之间的联系
ER relationship / relational relation	概念模型的联系 / 关系模型中的关系（表），不要仅因都译成 “关系” 而混淆

13. 复习清单与英文表达

以下是自我复习目标，不是正式模拟测试，也不代表官方分值。

- ☐ 区分实体、实体类型和实体集，并各举一个例子。
- ☐ 对给定属性分别判断复合性、多值性和派生性。
- ☐ 解释候选键的 “最小” 为何不等于 “属性数最少”。
- ☐ 给出弱实体的所有者、部分键、标识关系和完整标识。
- ☐ 对每条关系写出两个方向的最少与最多参与数。
- ☐ 正确使用键下划线、双矩形、双菱形、双椭圆、单双线。

☐ 说明为什么时间比例、借阅日期等可能是关系属性。
☐ 把图书馆图回读成需求，并识别重复借阅历史的模型边界。
☐ 对无法从题面确定的约束明确写假设，不自行当成已知。

可用于英文简答的表达：

> An entity type describes a collection of entities that share the same attributes.
>
> A candidate key is a minimal superkey: no proper subset is still a superkey.
>
> A weak entity is identified by its owner's identifier together with its partial key.
>
> Cardinality constrains the maximum number of related entities; participation specifies whether every entity must participate.
>
> Total participation requires every entity of that type to participate in at least one instance of the relationship.
>
> The attribute belongs to the relationship because its value describes a particular combination of participating entities.

来源与整理记录

- 教师课件：Week 1B - Conceptual DB Design.pptx，共 70 页；已读取文字，重点查看第 35–36、45、47、51、53、57、68 页图示。图书馆例子源于第 58–68 页，SVG 是学习用重绘，并省略非键实体属性。
- 范围依据：CSC5041_Teaching_Plan.pdf，第 1 页 Week 2、第 2 页 Tutorial 1；参考题面：Tutorial1.pdf，第 1 页 Question 1(a)–(b)。本篇未给出完整 Tutorial 答案或模拟卷。
- 对属性分类、弱实体部分键、关系角色、三元拆分及重复借阅边界的解释已在相应段落标明；不把解释性补充冒充教师原句。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入版本 SHA-256：ER 课件 `1f2bc2b2242208b0d3f0c650341521c65cba833cffe5da82fc27d0ddef7fc734`；Tutorial 1 `316e8b6bb9581a5ecae3fb2f449e395a4ea8a19b4ee16c4191fc20a6079d2c44`；syllabus 教学计划 `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`。这些是本轮读取版本，不沿用 01 的旧哈希。

03 关系模型与完整性约束：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

阅读说明与学习目标

本篇接续 02 ER 概念建模，对应教学计划第 1 页 Week 3 的 “The Relational Data Model” 与 “integrity constraints”。主要依据教师 Lecture2A.pptx 第 4–32 页；下文未注明文件的页码均按该 PPT 的幻灯片顺序计数。

学完后，应能读懂关系模式和实例，解释关系的集合性质，找出各类键及外键引用方向，区分键约束、实体完整性与参照完整性，并判断插入、删除、修改是否破坏约束。第 33 页开始的 ER 映射留到 04；关系代数留到 05；SQL 实现细节留到 06。

文中的学生选课数据是**自编教学示例**，不是真实学生资料、运行结果或考试题。对课件简化表述的辨析会明确标出。

1. 从 ER 模型走到关系模型

来源：第 4–7、32 页。

02 用实体、属性和关系表达应用含义；关系模型用一组相互关联的**关系 relation** 组织数据。这里的 relation 可直观理解为表，**不是 ER 图中连接实体的 relationship**。

数学术语	常见数据术语	含义
Relation 关系	Table 表	一组具有相同属性结构的元组
Tuple 元组	Row / record 行、记录	某组属性的一组取值
Attribute 属性	Column / field 列、字段	被命名的性质
Domain 域	Allowed values 允许值集合	该属性允许取得哪些值

例如 `Student(sid, name, email)` 是一个关系模式；`(S01, Amy, amy@example.test)` 是按该属性顺序书写的一个元组。Student 与 Course 之间的选课联系也可以由一个单独的关系 Enrolment 表示，不是靠把两张表放在相邻位置表示。

2. 模式、实例、度与基数

来源：第 8、32 页；以下形式化记法与 degree / cardinality 术语是解释性补充。

单个关系

```
关系模式：Student(sid, name, email)

一个关系实例：
sid   name   email
S01   Amy    amy@example.test
S02   Ben    ben@example.test
```

关系模式规定关系名、属性及其域；关系实例是某一时刻该结构下的实际元组集合。常写作 `R(A1, ..., An)` 与 `r(R)`，或在上下文清楚时简写为 `r`。

不考虑 NULL 的经典定义中，每个元组从各属性域各取一个值，因此 $r \subseteq \mathsf{dom(A1)} \times \ldots \times \mathsf{dom(An)}$。符号只说明 “每列从哪里取值、实例是哪些组合的集合”，不要求把所有可能组合都存进去。

术语	数什么?	上例
度 Degree / arity	属性数	3
基数 Cardinality	当前元组数	2

空实例仍有模式：Student 暂时没有记录时，度仍为 3，基数为 0。这里的 cardinality 是**当前关系有多少行**；02 的 ER cardinality ratio 是**业务规则允许关联多少个对方实体**，不要混用。

整个数据库

第 32 页将关系数据库模式定义为**一组关系模式及一组完整性约束**。因此仅列 Student、Course、Enrolment 的列名，还没有完整写出数据库模式；也要说明主键、外键等规则。

数据库状态包含各关系的实例；全部相关约束都满足时，才是合法状态。以下讨论的 “非法状态” 用来说明操作为何应被拒绝，并不表示系统应该保存它。

3. 一个关系的基本性质

来源：第 8–10 页；SQL 对照另见 SQLpart1 第 31–32 页。

性质	正确理解
元组无顺序	交换两行的显示位置，不会得到另一个关系
元组不能完全重复	经典关系是集合，相同元组不作为两个不同元素出现
每个属性位置取一个原子值	本课的平面关系模型不直接保留 ER 复合或多值属性
属性按名称区分	同一关系的属性名需要能区分各列；不同关系可有同名属性
重排显示列不改变含义	必须把列名和各自的值一起移动，不能只随意交换元组中的值

列无语义顺序，不等于元组值可以乱放

第 10 页将 PLAYER 的列连同列名与值一起重排，得到相同关系。用命名属性理解时，重要的是 “哪个值属于哪个属性”。

但在 `Student(sid, name)` 下，`(S01, Amy)` 与 `(Amy, S01)` 不一样：按位置书写元组时，位置必须与所声明的属性顺序对应。重排展示与交换属性值是两件事。

原子值与 ER 属性

02 中的 `address(city, street, postcode)` 是复合属性，`{phoneNumber}` 是多值属性。第 8 页明确排除在本课平面关系中直接使用这两种属性结构。原子性取决于当前模型所采用的域与操作，不是指一个字符串在字符层面无法拆开。具体怎样转换留到 04。

经典关系与 SQL 表

经典关系遵循集合语义；SQLpart1 第 31–32 页另指出 SQL 可以保留重复，查询结果可用 DISTINCT 去重。因此不能把 “关系无重复” 直接当作 “任何 SQL 表都自动禁止重复” 的结论。SQL 语法与具体约束声明留到 06。

4. 超键、候选键与主键

来源：第 12–15 页；与 02 第 4 节衔接。

键	条件	自编例子：假定 sid 与 email 分别唯一且必填
超键 Superkey	能唯一确定一个元组	`{sid}`、`{email}`、`{sid, name}`
候选键 Candidate key	是超键，且没有多余属性	`{sid}`、`{email}`
主键 Primary key	被选定的一个候选键	选择 `{sid}`
复合键 Composite key	由多个属性组成	Course 的 `{dept, courseNo}`，见第 8 节

最小性检查的是**删掉属性后还能否保证唯一**，不是 “在全体候选键中挑属性数最少的”。一张表可以有多个候选键，选定一个主键也不意味着其他候选键失去唯一性要求。

第 13 页以 “没有两人同名” 为**显式假设**，才把 PLAYER 的 Name 当候选键；不能从示例的三个人恰好不同名推导现实中姓名总是唯一。

为什么无重复元组还需要键约束？

```
(S01, Amy, amy@example.test)
(S01, Ada, ada@example.test)
```

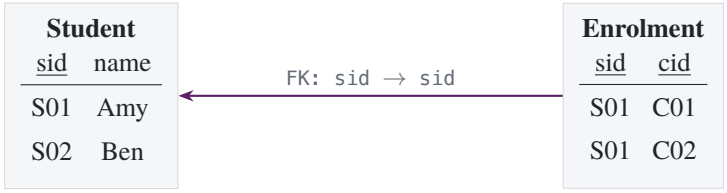
这两个完整元组不同，集合性质并不排除它们同时出现。但是它们具有相同 sid，若 sid 是主键，就违反键约束。**无完全重复行**与**指定键不能重复**是不同要求。

第 15 页的 `2^n − 1` 怎么理解？

该页列出所有非空属性子集。对 n 个属性，非空子集确实有 `2^n − 1` 个，**但不是每个子集都能唯一识别元组**，所以不能将它当作任意关系的超键数公式。

自编反例：R(A,B,C) 只有候选键 `{A}`，B、C 及 `{B,C}` 都不保证唯一。超键只有 `{A}`、`{A,B}`、`{A,C}`、`{A,B,C}`，共 4 个，不是 7 个。只有在每个单属性本身都能作键等额外条件下，所有非空子集才全是超键。

5. 外键：方向、重复值与复合引用



主键：唯一且非空
$PK(Student) = \{sid\}$

复合主键：整组唯一；sid 可以重复
$PK(Enrolment) = \{sid, cid\}$

$$\pi_{sid}(Enrolment) \subseteq \pi_{sid}(Student)$$

本例 sid 属于主键，故非空；一般外键是否可空取决于约束。

自编示例；仅展示 sid 引用。cid 还需引用 Course 的键。

来源：第 16–19 页。

外键 Foreign key 是引用方关系中的一个或一组属性，用其取值引用被引用关系的键。课件本节采用引用**主键**的定义；本篇按这一范围讲解，不把它扩展成所有 SQL 系统的完整规则。

```
引用方 referencing relation                被引用方 referenced relation
Enrolment.sid                  ————————→   Student.sid
Enrolment.(dept, courseNo)     ————————→   Course.(dept, courseNo)
```

核对外键时，要写清楚**引用方属性组 → 目标关系的键属性组**。不要求两边属性名相同，但对应域应兼容；仅仅都是字符串或列名相同，不能自动成立外键。

外键自身不要求唯一

多条选课记录可以有相同 sid，表示同一名学生选了多门课。Student.sid 必须唯一，Enrolment.sid 作为外键则可以重复。外键也可以同时是主键的一部分，这两个身份不冲突。

复合外键必须作为一个组合匹配

若 Course 中只有 `(CS,101)` 和 `(Math,201)`，那么 `(CS,201)` **不能**引用它们。虽然 CS 和 201 分别出现过，但它们没有在**同一个目标元组的对应键位置**共同出现。

课件第 19 页中的 `(Department, Degree)` 也是**一组复合外键**，不是 Department 和 Degree 各自单独引用 COURSE 的两个外键。遗漏任一部分，可能无法确定具体课程。

6. NULL 与外键的边界

来源：第 18、20、22、30 页；NULL 的未知/不适用含义已在 02 解释。

所在位置	本篇采用的规则
主键的任何组成属性	不能为 NULL，即使其他组成属性已有值

所在位置	本篇采用的规则
非主键属性	是否允许 NULL 取决于该属性的必填约束
可空外键	只有在其他约束也允许时，才可用 NULL 表示暂不引用
同时属于主键的外键属性	不能因为它也是外键就放宽主键非空规则

课件第 18、22 页的复合外键规则是：要么全部 NULL，要么全部非 NULL 且完整匹配目标键。本篇按这个明确给出的课堂规则判断，不将其泛化为所有 SQL 实现的默认行为。

自编小例子：给定 `Plan(planID, dept, courseNo)`，主键为 planID，`(dept,courseNo)` 是可空且遵守上述规则的外键，引用第 5 节的 Course。

外键值	判定
`(CS,101)`	合法，完整匹配
`(NULL,NULL)`	合法，假定两列都允许缺失且没有其他必填要求
`(CS,NULL)`	不合法，不符合本课 “全空或全非空” 的规则
`(CS,201)`	不合法，不存在这个组合

“参照完整性允许空引用” 不等于 “业务上允许没有关联”。若要求每条记录必须引用目标，还需要禁止外键为空。02 的全参与约束不能只靠 “有一个可空外键” 完整表达。

7. 三类完整性约束检查什么？

来源：第 20–22、32 页。

约束	要防止的问题	典型违规
键约束 Key constraint	不同元组共享同一个候选键值	两名学生使用相同 sid；或 sid 不同但唯一 email 重复
实体完整性 Entity integrity	主键不能完整识别一个元组	复合主键中某一属性为 NULL
参照完整性 Referential integrity	外键引用不存在的目标键	选课记录引用数据库中没有的学生

“主键唯一” 属于键约束，“主键非空” 属于实体完整性。它们共同支持标识，但答题时应分别说明。一个操作也可能同时违反多条规则；不要找到第一项就认定没有其他问题。

域约束和业务约束【解释性补充】

第 6、8 页的域规定值的范围。例如 grade 必须取规定等级或允许缺失，整数编号不能随意填自由文本。其他业务规则还可能要求费用非负、开始日期不晚于结束日期。

本篇三类约束是课件第 20 页重点列出的约束，**不是数据库全部可能约束的清单**。仅主键和外键全部合法，也不保证业务状态一定正确。

8. 贯穿例子：先明确模式、约束与初始状态

以下全部为自编教学例子。模式是已给定输入，不在这里演示 ER 映射。

模式与约束

关系模式	主键 PK	其他候选键	外键 FK
Student(sid, name, email)	{sid}	{email}	无
Course(dept, courseNo, title)	{dept, courseNo}	本例未指定其他候选键	无
Enrolment(sid, dept, courseNo, term, grade)	{sid, dept, courseNo, term}	本例未指定其他候选键	{sid} → Student.sid; {dept,courseNo} → Course.(dept, courseNo)

本例规定：Student 的三个属性都必填；课程三个属性都必填；sid、email、dept、title、name、term 是字符串，courseNo 是正整数；term 允许 2026F、2027S；grade 允许 A、B、C、D、F 或 NULL。NULL 表示尚未获得成绩。所有主键组成属性均非空。

一名学生可以选多门课，也可在不同 term 再选同一门课；**同一学生、同一课程、同一 term 最多一条选课记录**。这正是 Enrolment 四属性主键表达的规则。dept 或 courseNo 单独都不能唯一识别 Course。

初始状态

Student：

sid	name	email
S01	Amy	amy@example.test
S02	Ben	ben@example.test

Course：

dept	courseNo	title
CS	101	Database Basics
Math	201	Linear Algebra

Enrolment：

sid	dept	courseNo	term	grade
S01	CS	101	2026F	A
S02	CS	101	2026F	NULL

这个状态满足本例列出的约束：Student 两个候选键都唯一；所有主键完整非空；每个外键组合都能找到目标。Enrolment.sid 重复与否不决定它是否满足参照完整性；关键是它是否引用有效学生。

下文每个操作都从这个初始状态独立判断，不串联执行。以下是根据规则进行的静态推演，未在数据库中实际运行。

9. 插入：检查新元组与被引用目标

来源：第 21–28 页；以下操作使用第 8 节自编数据。

拟执行操作	判定与原因
向 Student 插入 `(S01, Ada, ada@example.test)`	不合法：sid 已存在，违反键约束
向 Student 插入 `(S03, Chen, amy@example.test)`	不合法：email 候选键重复，不能只检查主键
向 Enrolment 插入 `(S03, CS, 101, 2026F, A)`	不合法：Student 没有 S03，违反参照完整性
向 Enrolment 插入 `(S01, CS, 201, 2026F, B)`	不合法：Course 没有 `(CS,201)`；不能拆开匹配
向 Enrolment 插入 `(S01, CS, 101, NULL, B)`	不合法：主键组成属性 term 为 NULL，违反实体完整性
向 Enrolment 插入 `(S01, CS, 101, 2026F, B)`	不合法：四属性主键已存在，即使 grade 不同也不能再插一行
向 Enrolment 插入 `(S01, CS, 101, 2027S, NULL)`	合法：term 不同使主键不同，两组外键均有目标，grade 允许 NULL

答题顺序可以固定为：**域与必填要求** → **所有候选键** → **主键非空** → **每组外键**。题目要求列出全部违规时，需要查完所有相关规则。

对照课件原例：第 24 页向 RESEARCHER 插入已存在的 Person#=2，说明键冲突；第 26 页向 COURSE 插入 `(Comp.Sci., NULL)`，说明复合主键不能缺一部分；第 28 页用不存在的 Supervisee=6 插入 ENROLMENT，说明引用目标缺失。

10. 删除：沿外键反向查找引用者

来源：第 29–30 页。

判断删除时，不只看被删元组自身是否合法，还要看**有没有其他元组引用它的键**。

从初始状态拟删除什么?	直接删除的影响
Student 中 S01	有 Enrolment 引用 S01；若不处理这些引用，违反参照完整性
Course 中 `(CS,101)`	两条 Enrolment 引用它；若不处理引用，违反参照完整性
Course 中 `(Math,201)`	本例没有记录引用它，删除后仍满足已列约束
Enrolment 中 S01 的那条记录	本例没有别的关系引用 Enrolment，删除后仍满足已列约束

因此 “主键不能被删除” 不正确；真正要查的是依赖关系和已声明的处理规则。

第 30 页讨论的处理思路包括：

- **拒绝删除**：保留被引用元组，避免产生悬空引用。
- **级联删除引用者**：删除相关元组，但还要检查它们是否继续被其他关系引用。
- **将引用设为 NULL**：只在外键允许为空且不违反其他约束时可行。
- **改成另一个有效引用**：新值必须符合域及全部相关约束，不能随意写一个占位编号。

在本例中，Enrolment 的两个外键属性组都包含在主键中，因此**不能用设为 NULL 处理它们**。级联或改引用也必须符合业务含义；不能默认数据库自动选择一种方案。

11. 修改：同时考虑旧值与新值

来源：第 31 页；最后一段为对课件适用范围的解释。

修改位置	需要检查什么?	本例
主键或其他候选键	新值是否重复；主键是否变空；旧键是否仍被引用	把 Student S01 的 sid 改为 S02 会键冲突；改为 S03 又会让原 S01 引用失去目标，除非同时妥善处理
外键	新的完整组合是否能匹配目标；是否允许 NULL；是否兼任主键	把 S01 那条选课的 courseNo 从 101 改为 201，会形成不存在的 `(CS,201)`
非键属性	仍需检查域、必填和其他业务约束	把 grade 改为 Z，虽然没有改键，仍违反本例 grade 允许值规则

若将 S01 那条选课的 `(dept,courseNo)` 一起改为 `(Math,201)`，新的目标存在，且结果主键不与其他行重复，因此在本例约束下合法。**改变复合外键时，要检查变更后的整个组合。**

课件把修改主键类比为 “删除旧键再插入新键”，便于记住要查旧引用和新值冲突；这是约束分析方法，不要求真的先删除学生再重新插入。

课件措辞辨析：第 31 页 “Not unless you’re modifying the value of a key” 与 “all relational integrity constraints are to do with the key values” 应结合该页讨论的键相关约束理解。它们不能推广成 “所有非键更新都安全”：域、必填要求及业务规则同样可能被非键更新破坏。

12. 易混点速查

容易混淆	正确区分
Relation / ER relationship	元组集合 / 实体之间的联系
Schema / instance	结构和规则 / 某时刻的内容
Degree / cardinality	属性数 / 当前元组数
关系的 cardinality / ER cardinality ratio	实际行数 / 关联数量的业务上限
无完全重复元组 / 键唯一	整行不重复 / 指定属性组不能在不同行中重复
Candidate key / primary key	所有最小超键之一 / 被选定的一个候选键
主键 / 外键	标识本关系元组 / 引用目标关系的键；同一属性可兼任
外键值重复 / 悬空引用	多行引用同一目标通常允许 / 引用不存在的目标不允许
复合外键匹配 / 分别查每列	同一目标元组中整体匹配 / 分别存在不足以证明合法
Key constraint / entity integrity	唯一性 / 主键非空
NULL / 0 / 空字符串	缺失标记 / 一个数字值 / 一个字符串值
约束允许的状态 / 业务完全正确	满足已声明规则 / 还取决于是否把需要的业务规则声明完整

13. 复习清单与英文表达

以下是自我复习目标，不是正式模拟测试或官方分值安排。

- ☐ 从一组表中指出 relation、tuple、attribute、domain、schema 和 instance。
- ☐ 给出关系的 degree 与 cardinality，并解释空实例。
- ☐ 说明交换显示列与交换元组值的区别。

☐ 区分无重复行、超键、候选键和主键。
☐ 解释为何不能无条件使用 `2^n − 1` 计算超键数。
☐ 标出每组外键的引用方向，整体核对复合外键。
☐ 分别解释键约束、实体完整性、参照完整性。
☐ 根据课件规则判断外键全空、部分空及非空不匹配情况。
☐ 从同一初始状态独立判断插入、删除与修改，说明具体冲突元组或缺失目标。
☐ 说明级联删除与设为 NULL 的前提，不把它们当作自动行为。

可用于英文简答的表达：

> A relation is a set of tuples defined over a collection of named attributes.
>
> A relation schema describes the structure; a relation instance contains the tuples at a particular time.
>
> A candidate key is a minimal superkey, and a primary key is a selected candidate key.
>
> Entity integrity requires every component of a primary key to be non-NULL.
>
> Referential integrity requires a non-NULL foreign-key value to match a referenced key value.
>
> A composite foreign key must match the corresponding key components in the same referenced tuple.
>
> Deleting a referenced tuple requires checking the tuples that refer to it.

来源与整理记录

- 教师主课件：Lecture2A.pptx，主要第 4–32 页。已读取文字并查看第 7、10、15、17、19、24、26、28–29、31 页原图，核对表头下划线、复合外键与操作例子；第 33–55 页的映射步骤不在本篇展开。
- 补充对照：SQLpart1.pptx，第 31–32 页，仅用于区分经典集合语义与 SQL 重复值；不在此教授 SQL 查询。
- 范围依据：CSC5041_Teaching_Plan.pdf，第 1 页 Week 3。degree/cardinality、域约束与业务约束的解释、自编模式与数据、课件措辞的限定均已与原文区分。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入版本 SHA-256：Lecture2A `cecf28254342cf35e9a0b1b86a007fba728ed17a900e9e05182f2f2616081ee0`；SQLpart1 `aa28c989fb3674e780418c78c483bba20ff5a50b7da067b0341e60410a68c8ed`；syllabus 教学计划 `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`。

04 ER 到关系模式的映射：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

阅读说明与学习目标

本篇连接 02 ER 概念建模 与 03 关系模型与完整性约束：把 ER 图中的含义转成关系模式，并检查哪些约束已经保留、哪些仍需另行表达。

主要来源为教师 Lecture2A.pptx 第 33–56 页，对应教学计划第 1 页 Week 3 的 “ER-to-relational mapping”。下文未注明文件的页码均按该 PPT 的幻灯片顺序计数。Tutorial 1 第 1 页 Question 1(c)–(d) 分别要求写关系模式、说明哪些关系独立成表，以及识别仅靠键与外键不能表达的约束。

学完后，应能应用七步映射，明确主键、完整外键组、关系属性位置、唯一性与非空要求。本篇只写逻辑模式与约束，不提前讲 SQL 查询或规范化算法；例子是课件重述或明确标注的自编示例，不是数据库运行结果或整套 Tutorial 答案。

1. 映射的输入、输出与记号

来源：第 33、54–56 页；约束含义衔接 03。

```
输入：实体、属性、键、关系、基数、参与约束及文字业务规则
  ↓ ER-to-relational mapping
输出：关系模式 + 主键 + 外键组 + 唯一性/非空要求 + 其他约束说明
```

本篇使用下列可读性记号，**不是完整 SQL 建表代码**：

记号	含义
`R(a, b, c)`	关系 R 的属性列表
`PK{a,b}`	a、b 合在一起是一个复合主键，不是两个主键
`FK{a,b} → S{x,y}`	一组复合外键，a 对应 x、b 对应 y
`UNIQUE{a}`	非空的 a 值不能在不同行重复，用于保留相应唯一性
`NOT NULL{a,b}`	a 和 b 都必须有值

主键本身已要求唯一且每个组成属性非空。可空的 UNIQUE 记号在此只表达已存在关联不能重复，不规定各 SQL 产品如何处理 NULL；具体实现留到 06。

Tutorial 1(c) 要求**主键下划线、外键属性加星号**；正式作答应按题面标记，并另列复合外键的分组及目标。仅写几个星号不能说明哪些列合成一组引用。

2. 七步映射总览

1:N Department 1 — N Employee

Department(deptID, name)

Employee(empID, name, deptID)

$Employee.deptID \longrightarrow Department.deptID$

外键放 N 侧；若 Employee 全参与，再加 NOT NULL。

M:N Student M — N Course

Enrolment(sid, cid) $PK = \{sid, cid\}$

$Enrolment.sid \rightarrow Student.sid$ $Enrolment.cid \rightarrow Course.cid$

建立关联关系；关系属性也放在此处。外键本身不保证两端全参与。

自编简化示例；假设同一学生与课程最多关联一次。

来源：第 35、38、41、44、47、50、53–55 页。

课件步骤	输入结构	基本输出	主键/约束重点
1	强实体	每个强实体一个关系	保留选定实体键，展开复合属性
2	弱实体	弱实体关系，含所有者完整键	所有者键 + 部分键
3	二元 1:1 关系	通常把一端键加入另一端	外键还需唯一性；全参与端考虑非空
4	二元 1:N 关系	将 1 端键加入 N 端	N 端外键；关系属性也放 N 端
5	二元 M:N 关系	新建联系关系	两端完整键的组合通常作为主键
6	多值属性	新建属性关系	所有者键 + 属性值的各简单组成部分
7	n 元关系，n > 2	新建联系关系	先收集各参与者键，再按语义确定最小标识

顺序上先确定实体的完整标识，再处理普通关系和多值属性。**不是“每个菱形都必须建表”**，也不是“每个关系都只需一个外键”。唯一性、非空及覆盖要求必须结合原图回读。

独立联系关系中的每条元组都必须有全部参与者。各参与角色的键都应非空；已包含在主键中的部分自动非空，其余部分要明确声明。实体的部分参与通过没有对应联系行表示，不通过在一条联系行中把某个参与者写成 NULL 表示。

3. Step 1：强实体与普通属性

来源：第 35–37 页；派生属性含义见 02 的 ER 课件第 22–23 页。

对强实体 E 建关系 R：加入所有简单属性，将复合属性展开为简单组成部分；以 E 选定的键作为 R 的主键。其他候选键的唯一性也应保留，不能因选了一个主键就丢掉其他标识规则。

自编例子：Student 的键为 sid，email 也唯一；有复合属性 address(city,street,postcode)、多值属性 phone、存储属性 dateOfBirth 和派生属性 age。

```
Student(sid, name, email, city, street, postcode, dateOfBirth)
PK{sid}; UNIQUE{email}
```

若 email 在需求中必填，再明确非空。这里只处理普通属性：

- address 展开成三个属性，不同时再机械保留一个重复的 address 列。
- phone 不放进单个集合单元格，也不预设 phone1、phone2；Step 6 另处理。
- age 按本课 ER 例子的派生含义由 dateOfBirth 与参考日期计算，不默认重复存储。若另有物化需求，应作为额外设计选择说明。
- 若实体键本身是复合的，复制到其他关系时必须复制其全部组成属性。

4. Step 2：弱实体与标识关系

来源：第 38–40 页。

对弱实体 W 建关系：加入 W 的简单属性及复合属性的简单组成部分；加入所有者的完整主键作为外键；以**所有者主键 + W 的部分键**作主键。

```
Employee(empID, name)                          PK{empID}
Dependent(empID, dependentName, dateOfBirth, kinship)
  PK{empID, dependentName}
  FK{empID} → Employee{empID}
```

这是对课件 Dependent 例子的统一命名重述。dependentName 只在同一 Employee 内唯一，不能单独作 Dependent 的主键。empID 同时是主键组成部分和外键；其非空性与弱实体必须有所有者相对应。

标识关系通常不另建一个只重复这些键的关系。若标识关系还有属性，在本例 “每个弱实体恰好一个所有者” 的条件下，可随联系一起放到弱实体关系中。

所有者也是弱实体时

自编例子：Room 在 Building 内按 roomNo 编号，Desk 在 Room 内按 deskNo 编号。

```
Building(buildingID, name)                  PK{buildingID}
Room(buildingID, roomNo, floor)
  PK{buildingID, roomNo}; FK{buildingID} → Building{buildingID}
Desk(buildingID, roomNo, deskNo)
  PK{buildingID, roomNo, deskNo}
  FK{buildingID,roomNo} → Room{buildingID,roomNo}
```

Desk 对 Room 的引用是两属性外键；只复制 roomNo 会失去楼宇上下文。不要为 “看起来像弱实体” 而另外发明一个全局 ID，随后省略题目原有的局部唯一性要求。

5. Step 3：二元 1:1 关系

来源：第 41–43 页；以下显式唯一性与覆盖检查是对映射语义的补充。

设 S、T 是两端实体的关系。通常在其中一端加入另一端的完整主键作为外键，同时加入关系属性。课件建议：若一端全参与，优先把外键放在该端。

自编例子：每个 Department 必须由恰好一个 Employee 管理；一个 Employee 最多管理一个 Department。MANAGES 还有 startDate。

```
Employee(empID, name)                          PK{empID}
Department(deptID, name, managerID, managerStartDate)
  PK{deptID}
  FK{managerID} → Employee{empID}
  UNIQUE{managerID}; NOT NULL{managerID}
```

要保留的含义	哪条规则表达?
管理者必须是已存在的 Employee	外键
每个 Department 恰好有一个管理者	单值 managerID + 非空 + 外键
一个 Employee 最多管理一个 Department	managerID 唯一性
管理起始日期描述这组管理联系	放在携带管理外键的 Department 关系中

只有外键而没有 UNIQUE，会允许多个部门选择同一个管理员，退化成 N:1。第 41 页未展开所有约束声明，不能因此省略 1:1 的语义检查。关系属性是否必填要另看需求，不由“关系全参与”自动决定。

两端都是部分参与，或需要联系关系时

可以选一端放可空且唯一的外键，也可以单独建立：

```
Manages(deptID, empID, startDate)
PK{deptID}; UNIQUE{empID}; NOT NULL{empID}
FK{deptID} → Department{deptID}; FK{empID} → Employee{empID}
```

这表示每个部门最多一个管理者、每个员工最多管理一个部门；empID 非空保证每条已有管理联系确实有员工参与。但**没有要求所有 Department 都必须出现在 Manages 中**。若原图某端全参与，独立联系关系还需要该端的覆盖约束。

课件也提到在合适情况下合并双方，尤其双方全参与且不参加其他关系时。不要见到 1:1 就机械合表；应检查两类实体的标识、独立含义和其他关系是否仍能正确表示。

6. Step 4：二元 1:N 关系

来源：第 44–46 页。

把 **1 端的完整主键加入 N 端作为外键**；关系属性也放到 N 端。原有 N 端主键通常不变。

自编例子：Department:Employee = 1:N，每个 Employee 必须在一个 Department 工作，WORKS_FOR 有 joinedOn 属性。

```
Department(deptID, name)                    PK{deptID}
Employee(empID, name, deptID, joinedOn)
  PK{empID}; FK{deptID} → Department{deptID}
  NOT NULL{deptID}
```

- deptID 在 Employee 中允许重复，表示同一部门有多名员工；不要加 UNIQUE，否则会错误限制成 1:1。
- joinedOn 表示该员工与其所属部门的联系，放在携带外键的 Employee 中。
- Employee 全参与用 deptID 非空表达；若其可不属任何部门，则可以允许空引用，并说明没有联系时如何处理关系属性。
- 若 Department 端要求 “每个部门至少有一名员工”，仅 Employee.deptID 的外键与非空仍不足以保证；可能存在无人引用的 Department。

不要把员工编号列表塞进 Department 的一个单元格。一列不能直接保存本课平面关系模型中的多值集合。

7. Step 5：二元 M:N 关系

来源：第 47–49 页。

建立新的联系关系，加入两端完整键，各自作为外键，再加入关系属性。在普通 ER 关系的 “每对参与者对应一个联系实例” 语义下，使用两端键的组合作主键。

```
Employee(empID, name)                       PK{empID}
Project(projectID, title)                   PK{projectID}
WorksOn(empID, projectID, timeFraction)
  PK{empID, projectID}
  FK{empID} → Employee{empID}
  FK{projectID} → Project{projectID}
```

empID 单独不能作 WorksOn 主键，否则每人最多参加一个项目；projectID 单独也不行，否则每个项目最多一人。timeFraction 描述配对的性质，通常不放进主键，否则同一配对可因不同 timeFraction 重复出现。

只在两边实体各加一个外键，也不能表达任意 M:N 配对集合。联系关系用多行存储各个配对，避免把一方限制成单值联系。

同一配对允许多次发生怎么办？

若要记录同一 Reader 多次借同一 Book 的历史，应在概念模型中明确一次 Loan 事件及其标识。例如自编扩展：`Loan(loanID, readerID, bookID, startDate, endDate)`，主键为 loanID，两组外键分别为 `FK{readerID} → Reader{readerID}`、`FK{bookID} → Book{bookID}`，并要求 `NOT NULL{readerID,bookID}`，保证每次借阅都有读者和书。

新增事件标识应有需求依据；不能只为允许重复而把所有关系属性随意加入主键。普通 M:N 映射与重复事件建模的前提不同。

8. Step 6：多值属性

来源：第 50–52 页。

多值属性另建关系，包含实体完整键作为外键，以及属性值的简单组成部分。按课件的集合语义、且没有其他更小标识规则时，**所有者键 + 属性值**共同作主键。

多值简单属性

```
Department(deptID, name)                    PK{deptID}
DepartmentPhone(deptID, phone)
  PK{deptID, phone}; FK{deptID} → Department{deptID}
```

只用 deptID 作主键会使每个部门最多一部电话；只用 phone 会额外要求电话不能被不同部门共享，除非题目真的给出这一规则。没有电话的部门在 DepartmentPhone 中没有对应行，而不是填一行 `(deptID,NULL)`；phone 在此属于主键。

多值复合属性

自编例子：Person 有多个 Address，每个 Address 由 city、street、postcode 组成。

```
PersonAddress(personID, city, street, postcode)
PK{personID, city, street, postcode}
FK{personID} → Person{personID}
```

这里用完整地址值在同一人内部区分各地址，假定这些标识组成值完整可用，且没有额外更小的标识规则。同一人的同一地址不重复，符合多值属性是值集合的含义。若需要不完整地址、地址编号、标签或独立历史，应明确额外需求再调整设计，不机械套用此主键。

课件命名辨析：第 34 页多值属性写 Phone，第 52 页最终关系却写 D_number、Dnumber。本篇统一用语义清楚的 DepartmentPhone/phone 重述多值映射，不把这处命名变化当作新增业务属性。

9. Step 7：n 元关系与主键判断

来源：第 53 页。以下三元例子用于解释该页简略的键选择规则，不是课件原题。

对 n 元关系建立新关系，收集每个参与角色的实体完整键作为外键，加入关系属性。**键不等于“把所有列都圈上”，也不能只看哪个实体旁写了 1。**

没有额外唯一性限制的三元关系

```
Supply(supplierID, partID, projectID, quantity)
PK{supplierID, partID, projectID}
FK{supplierID} → Supplier{supplierID}
FK{partID} → Part{partID}
FK{projectID} → Project{projectID}
```

假定每组三者至多一条供应联系，且任意两个参与者都不能确定第三个，那么三方键组合作为主键，quantity 是关系属性。

若 “固定 Supplier 与 Part，最多对应一个 Project”

这条规则意味着 `(supplierID,partID)` 已能确定整个供应联系；假定没有其他更小唯一组，主键应选 `PK{supplierID,partID}`，projectID 仍是外键，但不必属于主键。此时还需显式加 `NOT`

NULL{projectID}，保证每条三元联系仍有 Project 参与；不能因它不在主键中就允许缺少该角色。

例如 (S1,P1,J1,10) 与 (S1,P1,J2,20) 若只用三方组合作键，会被视为两个不同键，从而没有阻止同一 Supplier–Part 对应两个 Project。**要保留这个“最多一个”，必须约束决定它的那组参与者。**

第 53 页的例外措辞没有展开“固定哪些角色后至多一个”的语义。本篇不据此推导“标 1 的那端键必定单独作主键”，而按明确的唯一性规则检查最小标识；存在多组候选键时，还应保留其他候选键的唯一性。

三元关系也不能不加判断就拆成三组二元联系，否则可能丢失“哪三个对象共同参与”的信息。题意不明确时，应列出需要确认的关联含义。

10. 贯穿例子：把课件公司 ER 图整理成模式

来源：第 34–52 页。以下保留课件实体与联系含义，但统一名称：SSN → empID，Department.Name → deptName，Pnumber → projectID，Dependent.Fname → dependentName。Department 仍按课件以名称为键，**没有额外发明 deptID**；这只是本例给定假设，不代表所有真实部门名称都唯一。

输入含义

- 强实体：Employee、Department、Project；弱实体 Dependent 由 Employee 与 dependentName 共同识别。
- WORKS_FOR：每位 Employee 在恰好一个 Department 工作，Department 可有多名员工。
- MANAGES：每个 Department 恰好一位管理者，Employee 最多管理一个 Department；关系属性 managerStartDate。
- WORKS_ON：Employee 与 Project 为 M:N，关系属性 timeFraction。
- Department 有多值 phone；Dependent 另有 dateOfBirth、kinship。

最终模式与键

关系模式	主键	外键
Employee(empID, firstName, lastName, dateOfBirth, deptName)	{empID}	{deptName} → Department{deptName}
Department(deptName, location, managerID, managerStartDate)	{deptName}	{managerID} → Employee{empID}
Project(projectID, projectName)	{projectID}	无
Dependent(empID, dependentName, dateOfBirth, kinship)	{empID,dependentName}	{empID} → Employee{empID}
WorksOn(empID, projectID, timeFraction)	{empID,projectID}	{empID} → Employee{empID}; {projectID} → Project{projectID}
DepartmentPhone(deptName, phone)	{deptName,phone}	{deptName} → Department{deptName}

还需明确：UNIQUE{Department.managerID}、NOT NULL{Department.managerID}、NOT NULL{Employee.deptName}。所有主键本身都非空。其他普通属性是否可缺失，按题目要求另定，不凭属性名猜测。

每条 ER 联系去了哪里?

ER 结构	映射落点	是否另建联系关系?
Employee—Dependent 的标识关系	Dependent.empID 及其复合主键	通常不另建
MANAGES，1:1	Department.managerID 与 managerStartDate，配合唯一性	不另建
WORKS_FOR，Department:Employee = 1:N	Employee.deptName	不另建
WORKS_ON，M:N	WorksOn	是
Department 多值 phone	DepartmentPhone	是，属于属性关系，不是新增 ER 二元关系

Employee 与 Department 互相引用不自动构成设计错误：两个方向表达不同业务联系。它们也**不会自动保证“管理者在自己管理的部门工作”**。若题目另加这项规则，还要明确表达，见下一节。

11. 映射后仍要检查的约束

来源：第 41、44、47、55 页的语义回读；应用参考 Tutorial 1 Question 1(d)。下列机制说明属于解释性补充，不是已经实现的数据库功能。

业务规则	仅有普通主键/外键是否足够?	还要表达什么?
1:1 中一名员工最多管理一个部门	不足，managerID 外键允许重复	对 managerID 的唯一性要求
携带外键的一端必须关联目标	可空外键不足	外键非空
每个部门至少一名员工	Employee 的外键即使非空仍不足	每个部门都被员工覆盖的全局存在性要求
M:N 关系中每个 Project 至少有一位参与者	WorksOn 两组外键和主键仍不足	每个 Project 都至少出现在一个 WorksOn 元组中
管理者必须在其管理部门工作	两条独立联系的键约束仍不足	管理配对必须属于工作配对的跨关系规则
startDate 不晚于 endDate	不足	值间比较约束

唯一性和非空可通过相应声明表达；日期比较可用检查约束。全局覆盖要求需要适合的全局检查或触发器等机制，不能用当前几条外键冒充已经保证。

同部门管理规则的一个精确表达：在第 10 节设计中，Department 的 `(managerID,deptName)` 必须出现在 Employee 的 `(empID,deptName)` 中。在支持相应复合键引用的实现中，可先声明 Employee 该属性组的唯一性，再增加这组复合外键；也可以由正确覆盖相关变更的触发器等机制检查。

这说明“现有模式中的单列外键不够”与“任何改造后的模式都绝不可能用复合外键表达”不是一回事。Tutorial 1(d) 作答应说清楚所采用的模式与机制，避免不加条件地声称某类约束永远不能由键/外键实现。

12. 一套可执行的答题顺序

来源：第 54–56 页；以下为整理后的工作流程。

1. **抄清输入约束**：每个实体键、弱实体所有者、关系两端的最小/最大参与数，以及额外文字规则。

2. **先建实体关系**：强实体展开普通属性；弱实体带上完整所有者键与部分键。
3. **处理普通联系**：1:1 选择承载端并补唯一性；1:N 放在 N 端；M:N、n 元另建关系。
4. **安置关系属性与多值属性**：前者随联系存储，后者独立成关系；不要漏复合属性的简单组成部分。
5. **逐表列主键和每组外键**：给出目标属性组，不能只标 “FK” 而不说指向哪里。
6. **回读最大约束和最小约束**：是否把 1:N 误限成 1:1？是否漏了 UNIQUE、非空或全参与覆盖？
7. **构造反例自查**：能否插入两个部门共用同一管理者？能否保留没有员工的部门？如果原规则禁止而模式允许，就还有约束没写。
8. **说明额外机制与假设**：将已由模式保证的规则与待实现的检查分开；按题面要求标下划线、星号等。

这些是模式层面的静态检查，不是 SQL 运行验证。不会因为列出了关系名，就宣称所有 ER 语义已自动保留。

13. 易混点速查

容易混淆	正确区分
强实体键 / 弱实体部分键	自身完整标识 / 需结合所有者上下文
复合外键 / 多个独立外键	一组属性共同引用一个目标键 / 各自有独立引用目标
1:1 的外键 / 唯一性	保证目标存在 / 限制该目标不能被多个源重复选用
N 端外键 / 1 端的编号列表	逐行保存单个归属 / 错把多值集合塞进一个属性
外键非空 / 目标端全参与	每个引用者有目标 / 每个目标至少有引用者
关系属性 / 新的标识属性	描述联系的性质 / 是否用于识别需要另有语义依据
M:N 联系关系 / 多值属性关系	保存实体配对 / 保存一个实体的多个属性值
三方键全组合 / 三元关系候选键	可区分三方组合 / 若有更强唯一性规则，完整组合未必最小
画出外键箭头 / 保留全部约束	引用方向明确 / 仍需检查唯一、非空、覆盖和其他业务规则

14. 复习清单与英文表达

以下是自我复习目标，不是正式模拟卷或官方分值安排。

- ☐ 不看笔记说出七步映射及各自输出。
- ☐ 把复合属性展开，并把多值属性和派生属性区别处理。
- ☐ 写出两层弱实体的完整主键和分组外键。
- ☐ 解释为什么 1:1 的外键通常还需唯一性。
- ☐ 判断 1:N 的外键与关系属性应该放在哪端。
- ☐ 为普通 M:N 关系选择组合主键，说明为何不随意加入关系属性。
- ☐ 映射简单及复合多值属性，并说明没有值时如何表示。
- ☐ 根据明确的三元唯一性规则选择最小标识，而非死记 1 端位置。
- ☐ 逐项回读贯穿例子的键、外键、唯一性与非空要求。
- ☐ 指出外键尚未保证的全参与或跨关系规则，并说明适用机制。

可用于英文简答的表达：

Map a strong entity type to a relation containing its simple attributes and the simple compo-

nents of composite attributes.

The primary key of a weak entity's relation combines the owner's primary key with the weak entity's partial key.

For a binary 1:N relationship, place the foreign key and relationship attributes in the relation on the N side.

A foreign key alone does not preserve a 1:1 relationship; uniqueness must also be enforced.

Map a many-to-many relationship to a separate relation containing the keys of both participants and the relationship attributes.

A non-NULL foreign key ensures that each referencing tuple has a target, not that every target is referenced.

For an n-ary relationship, choose the key from the stated uniqueness constraints rather than from the diagram labels alone.

来源与整理记录

- 教师主课件：Lecture2A.pptx，第 33–56 页；已读取文字，重点查看第 34、37、40、43、46、49–50、52–53、55 页原图，核对键下划线、外键箭头、参与线与多值属性命名。
- 范围依据：CSC5041_Teaching_Plan.pdf，第 1 页 Week 3、第 2 页 Tutorial 1；参考要求：Tutorial1.pdf，第 1 页 Question 1(c)–(d)。本篇未给出完整诊所题答案，未生成模拟题。
- Step 1 的派生属性解释衔接 02 中 ER 课件第 22–23 页；其余自编例子、统一命名、显式约束补充及课件简略规则的辨析已在相应位置说明。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入版本 SHA-256：Lecture2A `cecf28254342cf35e9a0b1b86a007fba728ed17a900e9e05182f2f2616081ee0`；Tutorial 1 `316e8b6bb9581a5ecae3fb2f449e395a4ea8a19b4ee16c4191fc20a6079d2c44`；syllabus 教学计划 `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`。

05 关系代数：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

阅读说明与学习目标

本篇接续 03 关系模型 和 04 ER 映射，主要依据教师 Relational_Algebra.pptx，共 61 页。下文未注明文件的页码均指该 PPT 的幻灯片顺序。

范围对应教学计划第 1 页 Week 4；Tutorial 2 第 1 页 Question 1(a)(i)–(vi) 涉及选择投影、连接、集合运算、未参加任何课程、所有目标课程以及分组聚合。聚合虽不是经典基本关系代数的一部分，但课件第 58–59 页和 Tutorial 均有覆盖，故本篇一起整理。SQL 留到 06。

学完后，应能解释每个运算的输入与输出模式，从自然语言组合表达式，并识别错误连接、过早投影、同名混淆和除法候选范围问题。自编数据和结果是静态教学推演，不是真实数据、数据库运行结果或新的模拟卷。

1. 关系代数的记号与闭包

来源：第 2–4、55–61 页。

关系代数以关系为输入，产生新的关系，结果可以继续参与运算，这称为**闭包 closure**。课件将它称为过程式 DML：表达式给出逻辑运算的组合，而不是数据库底层必须照做的物理执行顺序。

运算	本篇写法	主要作用
选择 Selection	$\sigma_{\text{condition}}(R)$	选行
投影 Projection	$\pi_{\text{attributes}}(R)$	选列并去重
重命名 Rename	$\rho_{\text{NewName(newAttributes)}}(R)$	区分关系角色或对齐属性名
并、交、差	$R \cup S$、$R \cap S$、$\texttt{R} - \texttt{S}$	合并、共有、左有右无
笛卡尔积 Product	$\texttt{R} \times \texttt{S}$	所有元组配对
条件连接 Theta join	$R \bowtie_{\text{condition}} S$	满足条件的元组配对
自然连接 Natural join	$R \bowtie S$	所有同名属性相等，合并这些属性
除法 Division	$R \div S$	与 S 中每个目标都关联的候选
分组聚合 Aggregation	$\gamma_{\text{groups; aggregates}}(R)$	每组统计

本篇用代码块中的 `{...}` 表示下标，便于在 Markdown 中稳定阅读；← 表示给中间关系命名，不修改原始数据。关系/属性重命名不改变已有值，完整列重命名列表按原属性顺序一一对应。

默认采用集合语义且例子不含 NULL。π 会消除重复；没有隐含行顺序。SQL 的重复值、NULL 和三值逻辑不能不加说明地搬进这些经典运算。Tutorial 中含 NULL 的 grade 不影响只使用其他属性的表达式；NULL 查询语义留到 06。

2. 贯穿例子：先写清输入

以下是自编的简化选课数据库，**不是 Tutorial 原模式**：省去 term，Takes 只表示学生是否选过某门课。

```
Student(sid, sname, major)          PK{sid}
Course(cid, title, dept, credits)   PK{cid}
```

```
Takes(sid, cid)                    PK{sid,cid}
Takes.sid → Student.sid; Takes.cid → Course.cid
```

Student：

sid	sname	major
S1	Amy	CS
S2	Ben	Math
S3	Amy	CS
S4	Dana	Math

Course：

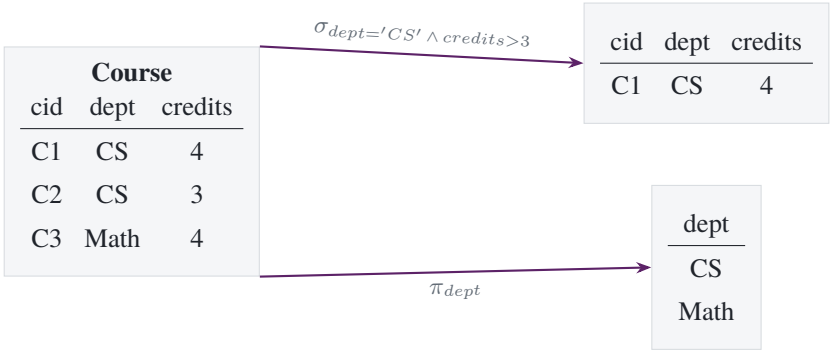
cid	title	dept	credits
C1	Databases	CS	4
C2	AI	CS	3
C3	Linear Algebra	Math	4

Takes：

sid	cid
S1	C1
S1	C2
S2	C1
S2	C3
S3	C3

S1、S3 同名，故**姓名不是标识**。S3 的专业是 CS，却只选了 Math 课程，故**学生专业与课程院系不是同一个条件**。S4 没有选课，后面判断 “从未” 和零记录情况时需要保留他。

3. 选择 σ：筛选元组，保留全部属性



选择：筛行，保留所有列。投影：选列，并消除重复。
采用第 05 章数据；为图示省略 title 列。

来源：第 5–10、18 页。

$\sigma_{\text{dept='CS'} \wedge \text{credits}>3}(\text{Course})$

结果模式仍是 `(cid,title,dept,credits)`，本例只含 `(C1,Databases,CS,4)`。选择不自动删列，也不修改被保留元组的值。条件可以比较属性与常量，或比较两个属性，并用 $\wedge$、$\vee$、$\neg$ 组合。

连续选择可合并，并可交换顺序：

$$\sigma_p(\sigma_q(R)) = \sigma_{p \wedge q}(R) = \sigma_q(\sigma_p(R))$$

前提是相关条件都对 R 的属性有定义。本例“CS 课程且超过 3 学分”可用两个连续选择实现；它不等于“CS 课程或超过 3 学分”。

不要混淆 SQL 的 SELECT 与关系代数的 selection：σ 选行，SQL 的 SELECT 列表主要决定输出列。

4. 投影 π：保留指定属性并消除重复

来源：第 11–18 页。

$\pi_{\text{dept}}(\text{Course}) = \{(\text{CS}), (\text{Math})\}$

虽然有两门 CS 课程，结果也只有一个 CS，因为结果是集合。投影后的模式只有 dept，cid、title、credits 已不在结果中。

将选择与投影组合：

$\pi_{\text{title}}(\sigma_{\text{dept='CS'} \wedge \text{credits}>3}(\text{Course}))$

结果模式为 `(title)`，本例是 `{(Databases)}`。从括号最内层向外读：先按条件筛课程，再取标题。

不能过早丢失后续需要的属性

正确：π_{title}(σ_{dept='CS'}(Course))

错误：σ_{dept='CS'}(π_{title}(Course))

第二式已经投影掉 dept，条件无法求值。一般而言，只有条件 p 用到的全部属性都保留在 X 中时，才可写：

$$\pi_X(\sigma_p(R)) = \sigma_p(\pi_X(R))$$

嵌套投影在 $X \subseteq Y$ 时满足 $\pi_X(\pi_Y(R)) = \pi_X(R)$。若外层想要内层已删除的属性，表达式没有定义，不是“结果为空”。

5. 重命名 ρ：处理角色和同名列

来源：第 55–56、60 页。

重命名可用于给集合运算对齐模式、区分自连接的两个角色，也能避免积和条件连接中的属性冲突。例如要找专业相同的不同学生对：

$A \leftarrow \rho_{A(\text{aid,aname,amajor})}(\text{Student})$

$B \leftarrow \rho_{B(\text{bid,bname,bmajor})}(\text{Student})$

$\text{Pairs} \leftarrow \pi_{\text{aid,bid}}(A \bowtie_{\text{amajor}=\text{bmajor} \wedge \text{aid}<\text{bid}} B)$

本例约定学号顺序 `S1<S2<S3<S4`，输出为 `{(S1,S3),(S2,S4)}`。`aid<bid` 同时排除自己和反向重复配对；若只用 $\text{aid} \neq \text{bid}$，会同时返回 `(S1,S3)` 与 `(S3,S1)`。

重命名只改变关系/属性标识，不是把 Amy 改成另一个姓名。不能把未重命名的 Student 与自身直接自然连接，当作所有不同学生配对；那会按所有同名属性相等来匹配。

6. 并、交、差：先保证模式相容

来源：第 19–32、56 页。

运算	保留哪些元组?	是否可交换?
$R \cup S$	在 R 或 S 中，包含两者都有的情况	是
$R \cap S$	同时在 R 和 S 中	是
`R − S`	在 R 中但不在 S 中	通常否

三者都要求 **union compatibility 并相容**。课件第 19 页采用同名且同域的一一对应属性要求；因此应先投影出相同属性组，必要时重命名并对齐。列数一样但含义/域不相容，不能直接相减。

用本例建立两个**学号集合**：

$\text{CSIDs} \leftarrow \pi_{\text{sid}}(\text{Takes} \bowtie \sigma_{\text{dept}=\texttt{'CS'}}(\text{Course}))$

$\text{MathIDs} \leftarrow \pi_{\text{sid}}(\text{Takes} \bowtie \sigma_{\text{dept}=\texttt{'Math'}}(\text{Course}))$

得到 `CSIDs={S1,S2}`，`MathIDs={S2,S3}`；以下简写省去单属性元组的括号。

表达式	本例结果	自然语言
$\text{CSIDs} \cup \text{MathIDs}$	`{S1,S2,S3}`	至少选过其中一类
$\text{CSIDs} \cap \text{MathIDs}$	`{S2}`	两类都选过
`CSIDs − MathIDs`	`{S1}`	选过 CS，但没选过 Math
`MathIDs − CSIDs`	`{S3}`	选过 Math，但没选过 CS

这里的 or 为包含两者都成立的 inclusive or。“只选其中一类”需另行表达，例如 $(\text{CSIDs} - \text{MathIDs}) \cup (\text{MathIDs} - \text{CSIDs})$。

“从未选课”不是一条记录上的不等条件

$\text{NeverIDs} \leftarrow \pi_{\text{sid}}(\text{Student}) - \pi_{\text{sid}}(\text{Takes})$

本例是 {S4}。先给出全部候选学生，再减去至少有一条选课记录的学生。不应以某条记录满足 cid ≠ C1 来表示“从未选 C1”：同一学生可能既选 C1，也选 C3。

7. 积与连接：从所有配对到正确配对

来源：第 33–47、61 页。

笛卡尔积 ×

R 的每一行与 S 的每一行配对。若 R 有 m 行、S 有 n 行，则积有 m×n 行；在属性名已消歧的前提下，度为两者度之和。本例 Student×Course 有 4×3=12 行、7 个属性。

积不会自动按主外键匹配。如果有同名属性，先用 ρ 消歧；本篇在条件连接中也用 关系名.属性名指明来源，可理解为对冲突属性作了相应限定重命名。

θ 连接与等值连接

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

该等式在属性名已消歧、条件有定义时成立。θ 可以是比较条件；若连接条件只使用相等比较，就是等值连接 equi-join。

$$\text{Takes} \bowtie_{\text{Takes.cid}=\text{Course.cid}} \text{Course}$$

这只配对课程编号相同的行。本例有 5 行。普通等值连接保留两边的连接属性副本（可用限定名区分）；**不会因两列值相等就自动把它们合并为一列**。

自然连接 ⋈

自然连接强制**所有同名属性**相等，并只保留这些属性的一份。不存在同名属性时，它退化为笛卡尔积。

$$\text{Student} \bowtie \text{Takes} \bowtie \text{Course}$$

在本例中，第一步只有 sid 同名，接着只有 cid 同名；结果模式为 (sid,sname,major,cid,title,dept,credits)，共 5 行。

自然连接不会理解“哪些列在业务上应该匹配”。若 Instructor 与 Course 都有 dept，自然连接可能额外要求教师院系等于课程院系；若两者不必相同，就应采用明确连接条件或先重命名无关的同名属性。

第 37 页把等值筛选后去掉一份连接列的结果联系到自然连接；严格使用时仍要按第 40–41 页检查所有同名属性，不能把任意“等值连接后投影”一概称为自然连接。连接只保留有匹配的组合，不自动给没有选课的 S4 补一行。

8. 组合查询：保留身份，最后输出姓名

来源：第 25–32、43–47 页；以下同名反例为自编。

找“既选过 CS 又选过 Math 的学生姓名”：先求学号交集，再连接 Student：

$$\text{BothIDs} \leftarrow \text{CSIDs} \cap \text{MathIDs}$$

$$\text{Answer} \leftarrow \pi_{\text{sname}}(\text{Student} \bowtie \text{BothIDs})$$

本例返回 {(Ben)}。若先分别取姓名，两边都含 Amy：CS 来自 S1，Math 来自 S3。姓名交集会错误返回 Amy，即把**两个不同的人**合成一个满足两条件的人。

对 “选过 CS 但没选过 Math” 也应先作学号差。本例正确者 S1 名叫 Amy，但两边姓名集相减会把 Amy 删掉。**投影对并可分配，但一般不能对交和差随意提前投影。**

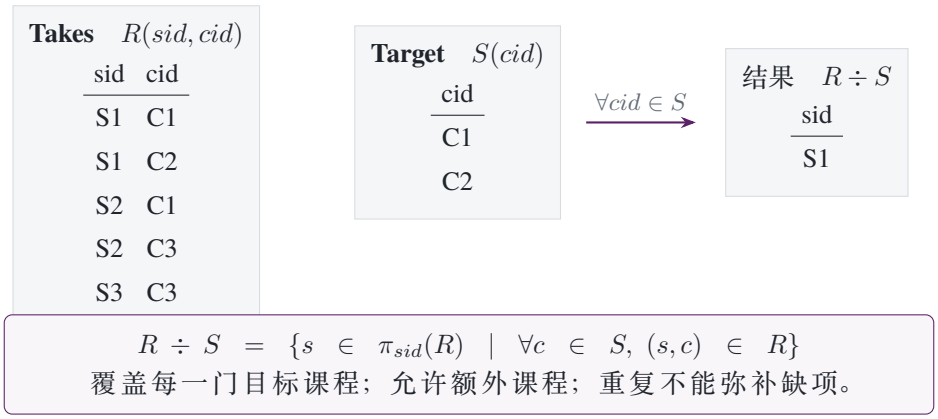
输出只要姓名时，最后的 π 会合并同名结果，这是题目要求的姓名集合；若需要区分学生，应保留 sid。不要用输出姓名这一要求反推姓名是键。

Tutorial 的 term 不能随意丢掉

Tutorial 2 的 Enrolled 和 Teaches 都包含 cid、term。“上过某教师教的课程” 需要同一课程、同一开课 term 对应；只按 cid 连接，可能把某学生在另一学期的选课错配给该教师。

本篇 Takes 特意省去 term，不能直接代替 Tutorial 的完整输入模式；套用模板前必须先核对关联所需的完整属性组。

9. 除法 ÷：表达 “每一个目标都满足”



沿用第 05 章数据。S2 缺 C2；S3 缺 C1、C2。
候选来自 $\pi_{sid}(R)$，因此 S4 不在除法的候选域内。

来源：第 48–54、61 页。

设 R 的属性为不相交的 X 与 Y，S 的属性恰为 Y。$R \div S$ 输出 X，保留那些在 R 中与 **S 的每个元组**都配对的 X 值，候选取自 $\pi_X(R)$。

$R(X, Y) \div S(Y) \to \mathsf{Result}(X)$

所以除法不是数值相除，也不是 “匹配其中任意一个”。需要先确定候选是什么 X、要求覆盖的目标是什么 Y。

本例：选过所有 CS 课程的学生

$R \leftarrow \pi_{\mathsf{sid,cid}}(\mathsf{Takes})$

$D \leftarrow \pi_{\mathsf{cid}}(\sigma_{\mathsf{dept='CS'}}(\mathsf{Course}))$

$\mathsf{AllCSIDs} \leftarrow R \div D$

D 为 `{C1,C2}`，S1 同时关联二者，S2 只关联 C1，S3 只关联 C3，故结果为 `{S1}`。最后可用 $\pi_{\mathsf{sname}}(\mathsf{Student} \bowtie \mathsf{AllCSIDs})$ 返回姓名。

- “所有 CS 课程” 要求集合应来自完整 Course，不应只从已有选课中提取，否则无人选过的必需课程会被漏掉。
- “所有 CS 课程” 允许额外选 Math 课程，不等于 “只选 CS 课程”。

- 被除关系只保留 (sid,cid)，不夹带 sname、term、grade 等无关属性，否则除法会保留它们并改变 “谁需要覆盖所有目标” 的含义。

多属性目标与结果模式

课件第 53–54 页有 R(A,B,C)、S(B,C)：

R ÷ S → 输出 A，要求覆盖 S 中全部 (B,C) 配对

R ÷ π_C(S) → 输出 (A,B)，要求每个固定 (A,B) 覆盖全部 C

两者的问题不同。按该页给定实例，第一式得到 {a1}；第二式得到 {(a1,b1),(a3,b2)}。不要只看到 “all” 就忽略目标关系的模式。

10. 除法的差集展开与空目标

来源：第 57 页；候选全集与空目标的讨论是定义边界的解释性补充。

把 “每个目标都有” 改写成 “没有缺失的目标”：

$$\text{Candidates} \leftarrow \pi_X(R)$$

$$\text{Required} \leftarrow \text{Candidates} \times S$$

$$\text{Missing} \leftarrow \text{Required} - R$$

$$\text{Bad} \leftarrow \pi_X(\text{Missing})$$

$$\text{Result} \leftarrow \text{Candidates} - \text{Bad}$$

这里 R 恰为 (X,Y)，Required 与 R 的模式需对齐。若原输入含更多属性，先投影到 X、Y。式中的 X,Y 是属性组，不是拿两个关系名作集合差。

在本例中，Candidates 为 {S1,S2,S3}。Required 有 6 个学号–课程配对；缺失 (S2,C2)、(S3,C1)、(S3,C2)，故 Bad 为 {S2,S3}，剩下 S1。R 中的额外 Math 配对不会造成缺失，因此不影响 S1 的资格。

目标为空时，必须说清候选范围

按上式，若 S 为空，Required 和 Missing 都为空，标准除法返回 $\pi_X(R)$。它**不会自动把从未在 R 中出现的学生补进来**。

如果题意是 “所有 Student 都作为候选；每人满足每个目标”，应使用明确全集 U：

$$U \leftarrow \pi_{\text{sid}}(\text{Student})$$

$$\text{Bad} \leftarrow \pi_{\text{sid}}((U \times D) - R)$$

$$\text{Result} \leftarrow U - \text{Bad}$$

这里 R、D 与第 9 节相同。D 非空时，此式与标准除法的结果一致（在本例外键有效的前提下）；D 为空时返回全部 U，包含没有选课的 S4。这体现空集上的全称条件为真，不代表任意实际题目都默认允许无目标；若题目排除这种情形，应说明假设。

11. γ：分组与聚合

来源：第 58–59 页；Tutorial 2 Question 1(a)(vi)。以下统一记号是本笔记的书写约定。

γ 可以配合 COUNT、SUM、AVG、MIN、MAX。$\gamma_{\text{分组属性; 聚合函数}\to\text{结果名}}(R)$ 表示每组产生一行，保留分组属性并新增聚合值。

$\text{Stats} \leftarrow \gamma_{\text{dept};\,\text{COUNT(cid)}\to\text{nCourses},\,\text{AVG(credits)}\to\text{avgCredits}}(\text{Course})$

dept	nCourses	avgCredits
CS	2	3.5
Math	1	4

再筛选课程数超过 1 的组：$\sigma_{\text{nCourses}>1}(\text{Stats})$，本例只剩 CS。聚合结果名要先在 γ 中定义，外层 σ 才能使用。

数的是哪一种对象?

- 本例 Course 一行一门课且 cid 唯一，因此 COUNT(cid) 数课程。
- 若先与 Takes 连接，同一课程可能按不同学生出现多行，直接计数可能数成选课配对，而非不同课程。
- 为数不同对象，应先在正确标识层面去重。例如从连接结果先投影 `(dept,cid,credits)`，再分组；不能仅投影 `(dept,credits)`，因为同院系两门相同学分的课可能被合并。
- $\gamma_{\text{sid};\,\text{COUNT(cid)}\to\text{nTaken}}(\text{Takes})$ 只产生 S1、S2、S3 的组，计数分别 2、2、1；不会自动产生 S4 的零计数组。

本例每个被聚合组非空且无 NULL。课件第 58 页提出 NULL 问题但未在此完整定义其语义；空输入或 NULL 的处理需明确约定，SQL 中的 COUNT、NULL 等细节在 06 处理。

12. 等价改写：先检查前提

来源：第 10、15–17、57、60 页；下表条件为相应定义的解释。

改写	成立前提或含义
$\sigma_p(\sigma_q(R)) = \sigma_{p\wedge q}(R)$	条件均有定义
$\pi_X(\pi_Y(R)) = \pi_X(R)$	$X \subseteq Y$
$\pi_X(\sigma_p(R)) = \sigma_p(\pi_X(R))$	p 用到的属性全部在 X 中
$R \cap S = R - (R - S)$	R、S 并相容
$R \bowtie_\theta S = \sigma_\theta(R \times S)$	属性已消歧，θ 有定义
$\pi_X(R \cup S) = \pi_X(R) \cup \pi_X(S)$	输入并相容且 X 可用，采用集合语义

一般不能把最后一式的 $\cup$ 直接换成 $\cap$ 或 $-$：投影会丢掉身份信息，第 8 节同名学生就是反例。连接中的提前投影也必须保留输出列、连接列、之后筛选所需列。

课件第 57 页给出基本运算 σ、π、$\cup$、$-$、$\times$、ρ，连接、交、除法可由它们导出。第 58 页的聚合则增加统计能力，**不是仅由这些经典基本运算直接获得的一般计数能力**；不要把“可导出的运算”与“加入更强表达能力的聚合”混为一谈。

13. 从英文需求写表达式的步骤

来源：第 25–32、43–54、60–61 页；下列为整理后的解题流程。

1. **先写输出模式**：要 sid、姓名、课程标题，还是每院系一行的统计?
2. **确定对象标识与候选范围**：同名对象用键区分；“未参加”需要完整候选全集。
3. **列完整连接条件**：根据题面核对主外键及 term 等上下文；不要只凭同名就自然连接。

4. **翻译量词与逻辑词**：at least one 通常用连接投影；or 用并；both 用交；never 用全集减存在；every 用除法或“减去缺失者”。
5. **保留中间需要的属性**：先过滤/连接/集合判断身份，最后投影输出展示属性。
6. **逐步写中间模式**：每个 σ 的条件列仍在吗？差集两边同模式吗？除法输出是不是所需候选？
7. **用小实例和反例检查**：同名、多个匹配、无匹配、额外课程、缺一门目标、目标为空。

Tutorial 2(a) 的六类题分别落在：选择投影、跨关系连接、并交、差集、除法、γ。这里给的是知识与方法；本篇没有逐题复写 Tutorial 的整套答案，也没有提前生成英文模拟卷。

14. 易混点、复习清单与英文表达

容易混淆	正确区分
σ / π	选行保留模式 / 选列并去重
无法求值 / 空结果	引用不存在属性属于表达式错误 / 合法表达式可能无匹配
并相容 / 列数相同	属性对应、名称和域按课件要求对齐 / 只有列数不足
or / only one of	包含同时满足 / 需要排除同时满足
等值连接 / 自然连接	明确相等条件且保留副本 / 所有同名属性相等且合并列
学生姓名 / 学生标识	可同名 / 用键追踪同一个对象
“有条记录不是” / “从未”	存在一个不满足的记录 / 不存在任何满足的记录
every / only	覆盖每个目标 / 排除目标之外的对象，两者不同
除法目标 / 除法候选	S 的全部元组 / 通常从 $\pi_X(R)$ 取得
COUNT 连接行 / COUNT 不同课程	配对次数 / 需要先保留课程标识并去重

以下是复习目标，不是正式模拟测试或官方分值：

- ☐ 写出每个运算的输入条件、输出模式与去重行为。
- ☐ 对本例手算选择、投影、学号并交差与自然连接。
- ☐ 正确使用重命名表达同表的两个角色。
- ☐ 用同名学生反例说明为何不能过早投影姓名。
- ☐ 解释 Tutorial 连接为何需要检查 term。
- ☐ 明确除法的 X、Y 和完整目标关系，并手算结果。
- ☐ 将除法展开成缺失配对的差集运算。
- ☐ 区分空目标下的标准除法与显式全集版本。
- ☐ 计算分组统计，识别连接重复和缺失零计数组。
- ☐ 对每条等价改写说明成立前提。

可用于英文简答的表达：

> Selection filters tuples while preserving the relation’s attributes.

> Projection retains the specified attributes and eliminates duplicate tuples.

> Union, intersection, and difference require union-compatible relations.

A natural join matches all common attributes and retains one copy of each common attribute.

Perform identity-based set operations before projecting names, because different entities may share a name.

Division returns candidates associated with every tuple in the divisor.

Universal conditions can be expressed by removing candidates that have a missing required pairing.

The object counted depends on the input tuples and the identifiers retained before aggregation.

来源与整理记录

- 教师主课件：Relational_Algebra.pptx，第 1–61 页文字已读；重点查看第 7、10、17、19、37–38、40、42、48、50、53–54、57、60–61 页原图/公式，核对选择投影条件、并相容、连接、除法与基本运算记号。
- 范围依据：CSC5041_Teaching_Plan.pdf，第 1 页 Week 4、第 2 页 Tutorial 2；参考题面：Tutorial 2.pdf，第 1 页 Question 1(a)(i)–(vi)，已查看原页及键下划线。教学计划写 Tutorial 2 为 Week 5，题面标题写 Week 6；本篇按实际题型整理，不用这处差异推断考试改期。
- 自编简化模式、姓名反例、空目标与候选全集说明、聚合书写约定及计数边界均已与教师原句区分；本篇不定义 SQL 的 NULL/重复值完整语义。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入版本 SHA-256：关系代数课件 `dcacfada73c2f482ab3b83ac7c8312cb2176932906b6d3e6cac071d01932283d`；Tutorial 2 `72c016526c1c0ff02c10fcb5f7f964c1184999c68422867102580093 6a6cabac`；syllabus 教学计划 `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354 ebeb68295`。

06 SQL 查询：中文学习笔记

CSC5041 · 第一次课堂测验复习 · 2026-10-19

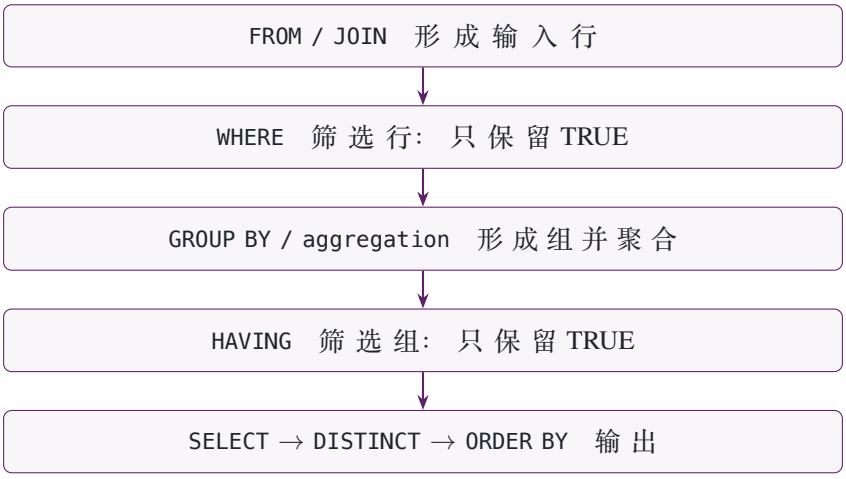
阅读说明与学习目标

本篇接续 05 关系代数，主要依据教师 SQLpart1.pptx 第 3–58 页，以及 SQLpart2.pptx 第 2–24 页中与查询相关的内容。下文用 P1/P2 区分两份课件，页码按幻灯片顺序计数。

教学计划第 1 页 Week 5 包含 SELECT–FROM–WHERE、排序、聚合、分组与嵌套查询；第 2 页 Tutorial 2 明列 GROUP BY、HAVING、NOT EXISTS、NOT IN with NULLs。因此不能只读文件名为 Part 1 的课件就停止。高级量化比较、DDL/DML、视图及 PL/pgSQL 的边界会另标，不把整份 Part 2 都认定为 Test 1 必考内容。

学完后，应能选择正确的结果粒度，写完整连接条件，处理重复值、NULL、分组、存在性和反存在性。所有例子使用自编数据，**查询结果是静态推演，未在 PostgreSQL 或其他数据库中执行**。补充官方文档用于核对语义，不决定教师考试深度。

1. 六个常用子句：书写顺序与逻辑顺序



普通查询的逻辑阶段，不是强制的物理执行顺序。
WHERE 不能直接使用同层的 AVG(score)；HAVING 可筛选聚合结果。

来源：P1 第 13、26–27、44–45 页；P2 第 18、22–24 页。

```
SELECT 输出列或表达式
FROM 输入表及连接
WHERE 行条件
GROUP BY 分组属性
HAVING 组条件
ORDER BY 排序属性;
```

WHERE、GROUP BY、HAVING、ORDER BY 按需要使用。P2 第 24 页展示的是本课查询框架，不是现代 SQL 所有语法的完整上限。

理解常规查询时可按以下**逻辑阶段**阅读：

```
FROM / JOIN → WHERE → GROUP BY / 聚合 → HAVING
            → SELECT → DISTINCT（若指定）→ ORDER BY
```

这不是优化器必须采用的物理执行顺序。WHERE 在组形成前筛行，不能在同一查询层直接用 `AVG(score)` 当行条件；输出别名也不要直接拿到同层 WHERE 中使用。ORDER BY 则可使用输出别名。

SQL 查询通常是声明式的：说明要什么结果。SELECT 中写一个计算表达式，只改变输出，不等于 UPDATE 修改原始数据。

2. 贯穿例子：在 05 的基础上加入学期与缺失成绩

以下为自编输入；**不是 Tutorial 原模式**，也不沿用 05 的 Takes 作为完整选课表。

```
Student(sid, sname, major)                PK{sid}
Course(cid, title, dept, credits)         PK{cid}
Enrolment(sid, cid, term, score)          PK{sid,cid,term}
Enrolment.sid → Student.sid; Enrolment.cid → Course.cid
```

除 score 外各列均必填；score 是可空数值，NULL 表示成绩尚未获得，不是 0。term 是字符串。

Student：

sid	sname	major
S1	Amy	CS
S2	Ben	Math
S3	Amy	CS
S4	Dana	Math

Course：

cid	title	dept	credits
C1	Databases	CS	4
C2	AI	CS	3
C3	Linear Algebra	Math	4

Enrolment：

sid	cid	term	score
S1	C1	2026F	90
S1	C2	2026F	80
S1	C1	2027S	NULL
S2	C1	2026F	NULL
S2	C3	2026F	80

sid	cid	term	score
S3	C3	2026F	NULL

共有 6 次选课、3 个非空成绩、2 种不同非空成绩（80、90）。S1 重修 C1；S1 与 S3 同名；S4 无选课。后续除明确的假设变更外，都独立使用这份输入。

3. 基本查询、别名、字符串与 DISTINCT

来源：P1 第 12–13、18–25、28–40 页。

```sql
SELECT c.cid, c.title, c.credits * 2 AS doubled_credits
FROM Course AS c
WHERE c.dept = 'CS' AND c.credits > 3;
```

结果为 `(C1,Databases,8)`，Course 中的 credits 仍是 4。字符串用半角单引号；字符串内的单引号加倍，例如 `'O''Brien'`。不要把中文弯引号从 PPT 直接复制到代码中。

写法	含义
`=、<>、<、<=、>、>=`	比较运算；本篇用 <> 表示不等
`IN ('CS','Math')`	属于给定值列表
`LIKE 'A%'`	以 A 开头，% 可匹配零个或多个字符
`LIKE '_e%'`	第二个字符是 e，_ 匹配一个字符
`BETWEEN 3 AND 4`	含边界的范围比较

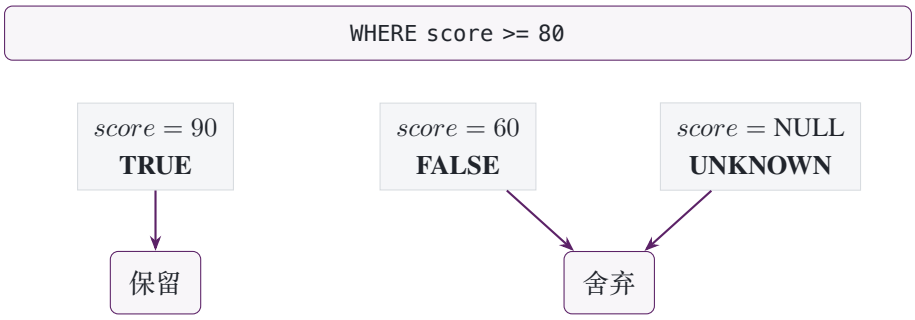
BETWEEN 是补充写法。混用 AND、OR 时主动加括号，例如 `(major='CS' OR major='Math') AND sid<>'S4'`，避免把组合条件读错。

SQL 默认保留输出重复

```sql
SELECT s.sname FROM Student AS s;
SELECT DISTINCT s.sname FROM Student AS s;
```

第一式输出 Amy 两次、Ben 一次、Dana 一次；第二式只有三种姓名。没有 ORDER BY 时不要假定这些行按某个顺序返回。`DISTINCT sid,sname` 对**整个输出属性组**去重，不是单独对每列去重。只要输出姓名，不表示姓名是键；连接与集合判断仍应优先保留 sid。

4. NULL 与三值逻辑



FALSE 与 UNKNOWN 是不同逻辑值，但 WHERE 都不保留。
检查空值应使用 IS NULL，而非 = NULL。数值为自编示例。

来源：P1 第 18、35–36 页；补充核对 PostgreSQL 比较运算文档。

普通比较遇到 NULL 通常得到 **UNKNOWN**。WHERE 只保留条件为 TRUE 的行；FALSE 与 UNKNOWN 都不保留，但二者不是同一个逻辑值。

表达式	结果
`NULL = NULL`、`NULL <> NULL`、`NULL > 80`	UNKNOWN
`NOT UNKNOWN`	UNKNOWN
`TRUE AND UNKNOWN`	UNKNOWN
`FALSE AND UNKNOWN`	FALSE
`TRUE OR UNKNOWN`	TRUE
`FALSE OR UNKNOWN`	UNKNOWN

课件措辞辨析：P1 第 18 页的 “Comparison against NULL yields FALSE” 不应按字面记成逻辑定律。第 35 页明确给出 UNKNOWN；“被 WHERE 过滤掉” 不等于 “表达式本身为 FALSE”。

```sql
SELECT sid, cid, term
FROM Enrolment
WHERE score IS NULL;
```

得到 `(S1,C1,2027S)`、`(S2,C1,2026F)`、`(S3,C3,2026F)`，结果无保证顺序。不能用 `score = NULL` 替代；也不能用 `score <> 90` 找出未出分记录，因为这些记录的比较结果是 UNKNOWN。

若要 “不是 90 分，或者未出分”，明确写 `score <> 90 OR score IS NULL`。数值 0 是一个已知成绩，不应无需求依据地把 NULL 改成 0。

5. 多表连接：先确定完整配对条件

来源：P1 第 41–49、52–58 页。

```sql
SELECT s.sid, s.sname, c.title, e.term
FROM Student AS s
JOIN Enrolment AS e ON e.sid = s.sid
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'CS';
```

本例得到 4 行：S1 的 C1 两个学期、C2 一次，以及 S2 的 C1 一次。若只输出姓名，会有三行 Amy、一行 Ben；题目要求不同学生时，可输出 `DISTINCT s.sid,s.sname`，而不是掩盖错误连接。

遗漏连接条件可能产生笛卡尔积；两张有 name 的表写 `name=name` 还可能因歧义报错。用表别名限定列名，也让每条联系更容易检查。

NATURAL JOIN 的风险

它匹配**所有同名列**并合并它们。P1 第 55–57 页说明教师院系与课程院系未必相同，不能无意增加相等条件。优先用清楚的 JOIN…ON 表达已确认的业务联系。

Tutorial 2 的 Enrolled 与 Teaches 都有 cid、term，教师与选课的关联要同时匹配二者：`e.cid=t.cid AND e.term=t.term`。只按 cid 会把不同学期混在一起。本篇没有 Teaching 表，因此不把该条件当作可直接对本例运行的完整查询。

对 INNER JOIN，某些条件可在 ON 与 WHERE 间等价调整；不要把它理解为固定的物理执行先后，也不要推广为所有外连接都等价，外连接会涉及补 NULL 的行。

6. 自连接与排序

来源：P1 第 50–51 页；P2 第 23 页。

找专业相同的不同学生配对：

```sql
SELECT a.sid AS sid1, b.sid AS sid2
FROM Student AS a
JOIN Student AS b ON a.major = b.major
WHERE a.sid < b.sid
ORDER BY sid1, sid2;
```

按本例学号的通常字符顺序，结果为 `(S1,S3)`、`(S2,S4)`。`<` 排除自己和反向重复；只有 `<>` 会保留正反两种顺序。

```sql
SELECT cid, title, credits
FROM Course
ORDER BY credits DESC, cid ASC;
```

结果顺序为 C1、C3、C2。只按 credits 排序时，C1 与 C3 同为 4，二者先后未确定；若题目要求稳定先后，要增加明确的次级排序键。排序不等于去重，也不改变数据在逻辑关系中的身份。

7. 聚合：COUNT、SUM、AVG 与 NULL

来源：P2 第 12–16 页；补充核对 PostgreSQL 聚合文档。

```sql
SELECT COUNT(*) AS n_rows,
       COUNT(score) AS n_scored,
       COUNT(DISTINCT score) AS n_distinct_scores,
       SUM(score) AS total_score,
       AVG(score) AS mean_score
FROM Enrolment;
```

本例为 (6,3,2,250,250/3)，平均值约 83.33，显示精度取决于数值类型。

写法	数什么或如何处理 NULL?
COUNT(*)	输入行数，包括 score 缺失的行
COUNT(score)	score 非空的行数，保留重复值的次数
COUNT(DISTINCT score)	不同非空 score 值的个数
SUM/AVG/MIN/MAX(score)	基于非空输入值计算

AVG 的分母是本例 3 个非空成绩，不是 6 次选课。AVG(DISTINCT score) 则为 (80+90)/2=85，已经改变统计问题，不是一般的去重修复。

空输入与全 NULL

无 GROUP BY 的汇总在没有匹配行时仍产生一行：COUNT 为 0，SUM、AVG、MIN、MAX 为 NULL。若匹配行存在但 score 全为 NULL，COUNT(*) 仍数这些行，COUNT(score)=0，其他四种 score 聚合仍为 NULL。

例如筛选 sid='S3' 后，COUNT(*)=1、COUNT(score)=0、AVG(score)=NULL。NULL 不是自动补零；是否把缺失汇总显示成 0，需要业务含义支持。

8. GROUP BY 与 HAVING：每组一行

来源：P2 第 17–22 页。

```sql
SELECT e.sid, COUNT(*) AS n_enrolments,
       COUNT(e.score) AS n_scored, AVG(e.score) AS mean_score
FROM Enrolment AS e
GROUP BY e.sid
HAVING COUNT(*) >= 2
ORDER BY e.sid;
```

sid	n_enrolments	n_scored	mean_score
S1	3	2	85
S2	2	1	80

WHERE 筛选输入行，HAVING 筛选已经形成的组。若先加 WHERE score IS NOT NULL，再 COUNT(*)，数的就是有成绩的记录，S2 不再满足至少两行，不能声称与原查询相同。

按本课可直接采用的书写规则：SELECT 中非聚合的列都明确列入 GROUP BY；不要查询 sid, cid,AVG(score) 却只按 sid 分组，从一个学生的多门课中任意挑 cid。某些实现存在基于函数依赖的例外，不影响先按这条规则写清楚的做法。

统计对象与零记录

本例 S1 有 3 次选课，却只有 2 门不同课程；数课程可用 `COUNT(DISTINCT cid)`。连接 Enrolment 后统计课程数量时，同一课程也可能被多名学生重复计入。必须先回答“数选课记录，还是数不同课程”。

本查询不会为无选课的 S4 生成零计数组。若需要所有学生含零计数，须另设计覆盖全部学生的查询，通常涉及外连接；不能用当前内连接/分组结果冒充已经包含 S4。

9. 子查询：先区分标量与多行结果

来源：P2 第 2–5 页；基数边界补充见 PostgreSQL 子查询文档。

标量子查询

```
SELECT cid, title
FROM Course
WHERE credits > (SELECT AVG(credits) FROM Course);
```

总体平均学分是 11/3，结果包含 C1、C3。这里的内层聚合保证一列一行，适合当一个值使用。

标量位置要求一列且至多一行：零行相当于 NULL，多于一行在 PostgreSQL/标准标量语义下是错误，不是自动取第一行。例如把内层替换成 `SELECT credits FROM Course WHERE dept='CS'` 会返回两行，不能直接当标量比较。这个 AVG 子查询不需要引用外层别名。

IN：测试一个值是否在结果中

```
SELECT s.sid, s.sname
FROM Student AS s
WHERE s.sid IN (
    SELECT e.sid
    FROM Enrolment AS e
    WHERE e.score >= 85
);
```

本例只有 S1。单值 IN 的子查询应返回一列；右侧重复值不会让同一外层行重复输出。IN 的逻辑与 NULL 的关系在第 11 节说明。

10. EXISTS、NOT EXISTS 与相关子查询

来源：P2 第 6 页；Tutorial 2 主题列有 NOT EXISTS。

相关子查询引用外层当前行，例如内层的 `e.sid=s.sid`。可按“对每个外层学生问一个存在性问题”理解，不代表数据库一定逐行执行完整子查询。

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE EXISTS (
    SELECT 1
    FROM Enrolment AS e
    WHERE e.sid = s.sid AND e.score IS NULL
);
```

本例返回 S1、S2、S3。EXISTS 关心子查询是否有行，不关心 SELECT 列表中的 1；这个普通存在性查询即使有多个匹配，也只为每个外层学生返回一次。外层自己若已有重复行，EXISTS 不会额外替它去重。

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM Enrolment AS e
    WHERE e.sid = s.sid
);
```

本例只有 S4。“从未” 要检查**没有任何匹配行**，不是找一行 `cid <> 'C1'`。若忘记相关条件，内层只要全表存在任意记录，就会影响所有外层学生。

11. NOT IN 遇到 NULL：反例要会解释

来源：教学计划第 2 页 Tutorial 2；补充依据 PostgreSQL IN/NOT IN 说明。

对非空数值，`80 NOT IN (90,NULL)` 等价于 `80<>90 AND 80<>NULL`，结果为 UNKNOWN，WHERE 不保留它；`90 NOT IN (90,NULL)` 则因已找到相等值而为 FALSE。**不是任何含 NULL 的 NOT IN 都产生同一个结果。**

情况	单值比较的结果
右侧存在相等值	IN 为 TRUE，NOT IN 为 FALSE
没有相等值，但比较涉及 NULL，且右侧非空	可能为 UNKNOWN，不能当作确定 “不在”
右侧为空集	IN 为 FALSE，NOT IN 为 TRUE

对本例这个 “找无选课学生” 的特定问题，下面写法安全：

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE s.sid NOT IN (SELECT e.sid FROM Enrolment AS e);
```

原因是两边 sid 都保证非空；本例仍返回 S4。若换成可空列，这个推理就不成立。删除右侧 NULL 也不能自动解决左侧 NULL 的含义；使用相关 NOT EXISTS 时，同样要明确 NULL 键应如何匹配，不能声称它与 NOT IN 在任意空值条件下都等价。

12. UNION、INTERSECT、EXCEPT 与身份粒度

来源：P2 第 9 页；与 05 的集合运算对应。

```sql
SELECT e.sid
FROM Enrolment AS e
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'CS'
INTERSECT
SELECT e.sid
FROM Enrolment AS e
JOIN Course AS c ON c.cid = e.cid
WHERE c.dept = 'Math';
```

结果为 S2。两侧课程条件不要求同一条选课记录同时属于两个院系；它们先分别求学生集合，再按身份求交。

运算	默认行为
UNION	合并并去除输出重复
UNION ALL	合并并保留重复次数
INTERSECT	共有元组，去重
EXCEPT	左边有、右边没有的元组，去重，方向重要

输入列数必须一致，对应位置类型要相容；SQL 集合运算按列位置对应，不要求像本课命名关系代数那样两边列名完全相同。INTERSECT ALL、EXCEPT ALL 另有多重集次数语义，基础复习先与默认去重版本区别，不把 ALL 理解成无作用的装饰。

若先在两边选 sname 再求交，会让 S1 的 CS 记录与 S3 的 Math 记录因同名 Amy 被错误合并；应先求 sid，再连接或用 IN/EXISTS 取得姓名。

13. “每个目标都满足”：没有缺失目标

来源：P2 第 10–11 页；下例为将 05 方法写成 SQL 的自编解释，不是 Tutorial 逐题答案。

找选过每门 CS 课程的学生，可以从 Student 候选出发，排除存在某门 CS 课却找不到该学生选课记录的情况：

```sql
SELECT s.sid, s.sname
FROM Student AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM Course AS c
    WHERE c.dept = 'CS'
      AND NOT EXISTS (
          SELECT 1
          FROM Enrolment AS e
          WHERE e.sid = s.sid AND e.cid = c.cid
      )
);
```

本例只有 S1。内层寻找某个学生–课程配对；中层寻找缺失配对的目标课程；外层要求这样的目标不存在。跨学期选过即可，是本例明确采用的含义；若要求某个 term，须在目标与选课条件中明确对应的范围。

这要求覆盖所有目标，允许额外选非 CS 课程；重复选同一门课也不能代替缺失的另一门课。目

标来自完整 Course，不能仅从已有人选过的课程产生。

若假设数据库中没有 CS 课程，此式返回所有 Student，包括 S4，因为不存在可缺失的目标。若题意要求至少有一个目标，要另加目标非空要求。课件 P2 第 11 页从 Sells 中枚举候选，空目标时只返回在 Sells 中出现过的 bar；这与本例从完整 Student 枚举候选的范围不同，应联系 05 的空除数讨论。

14. 课件其余内容与本轮范围

ANY / ALL【补充辨识】

来源：P2 第 7–8 页。教学计划 Week 6 列高级子查询及集合比较，本节只帮助识别记号，不据此新增 Test 1 深度要求。

`x > ANY (subquery)` 问是否至少一个比较为真；`x > ALL (subquery)` 问是否所有比较都为真。空右侧时 ANY 为 FALSE、ALL 为 TRUE。含 NULL 时还可能 UNKNOWN；不能无条件将 ALL 改写成与 MAX 比较，因为空集或 NULL 可使语义不同。

DDL、数据修改、视图与 PL/pgSQL

P2 第 25–52 页还讲 CREATE TABLE、主外键、INSERT、DELETE、UPDATE、视图和结构修改。这些可联系 03–04 理解，但本章不据文件位置将它们全部算作 Test 1 查询重点：

- PRIMARY KEY 结合唯一性与非空；FOREIGN KEY 并不自动要求该列非空，1:1 也不能仅靠一个普通外键保证。
- DELETE 删除记录，DROP TABLE 删除表结构；UPDATE 没有 WHERE 会作用于全表。这里只解释命令，不执行任何修改。
- 普通视图是查询定义的接口，不应将课件 “does not take up space” 绝对理解为连定义元数据都不占空间，也不等于物化视图保存结果。
- PL/pgSQL、索引、事务、规范化等超出本篇，不展开。

15. 写查询的检查顺序与复习清单

1. **确定输出粒度**：每个学生、每次选课、不同课程，还是每组一行？
2. **先核对模式和完整连接键**：尤其 sid、cid、term；区分学生专业与课程院系。
3. **写行条件并处理 NULL**：WHERE 只留 TRUE；选择 IS NULL、IN、EXISTS 等合适形式。
4. **明确重复值要求**：需要 DISTINCT 吗？DISTINCT 是在掩盖错误连接，还是符合输出粒度？
5. **分组后再筛组**：SELECT 中的普通列明确分组；COUNT(*)、COUNT(col)、COUNT(DISTINCT col) 不混用。
6. **检查子查询形状与相关条件**：标量是否至多一行？存在性是否真的关联当前外层对象？
7. **检查边界**：同名、重修、未出分、无匹配、重复匹配、空目标；需要顺序时加 ORDER BY。

易混点	正确区分
UNKNOWN / FALSE	逻辑不同；WHERE 都不保留
DISTINCT / GROUP BY	输出元组去重 / 形成统计组，不可机械互换
WHERE / HAVING	组形成前筛行 / 聚合后筛组
COUNT(*) / COUNT(score)	所有输入行 / 非空 score 行
EXISTS / 标量子查询	是否有行 / 把结果作为一个值
NOT IN / NOT EXISTS	比较值集合 / 检查相关匹配；NULL 下不无条件等价
“所有” / “只有”	覆盖每个目标 / 排除目标之外的联系

易混点	正确区分
SQL 集合运算 / 普通 SELECT	默认去重 / 默认保留重复

以下是复习目标，不是正式模拟卷或官方分值：

- ☐ 不看笔记写出常用子句的书写顺序与逻辑阶段。
- ☐ 手算基本查询、DISTINCT、自连接及带次级键的排序。
- ☐ 用三值逻辑解释为什么未出分记录不满足 `score<>90`。
- ☐ 检查普通连接和跨学期连接的完整条件。
- ☐ 手算三种 COUNT、全 NULL 与空输入汇总。
- ☐ 解释 WHERE 与 HAVING 改变统计结果的方式。
- ☐ 区分标量、IN、EXISTS 子查询的结果要求。
- ☐ 解释 NOT IN 含 NULL 的反例与非空 sid 的安全情形。
- ☐ 用学生身份作集合运算，并解释不能过早投影姓名。
- ☐ 读懂双重 NOT EXISTS 的三层含义及空目标行为。

可用于英文简答的表达：

> WHERE retains only rows for which its condition evaluates to TRUE.
>
> COUNT(*) counts rows, whereas COUNT(column) counts non-NULL values in that column.
>
> WHERE filters input rows; HAVING filters groups after aggregation.
>
> EXISTS tests whether a subquery returns any rows, not the values in its SELECT list.
>
> A NOT IN comparison can evaluate to UNKNOWN when NULL participates and no equal value is found.
>
> A universal requirement can be expressed as the absence of a required target without a matching association.

来源与整理记录

- 教师课件：SQLpart1.pptx、SQLpart2.pptx。本篇核心来源为 P1 第 3–58 页、P2 第 2–24 页；已读取文字并重点查看 P1 第 18、31、35–36、51、55、58 页，P2 第 2、6、9、11、15–16、20、22、24 页原图。P2 第 25–52 页仅作范围辨识。
- 教学范围：CSC5041_Teaching_Plan.pdf，第 1 页 Weeks 5–6、第 2 页 Tutorial 2；参考题面：Tutorial 2.pdf，第 1 页。题面 Week 6 与计划 Week 5 标签差异已在 05 记录，本篇不推断改期。
- 补充核对：PostgreSQL 18 官方文档的 比较、子查询、聚合、表表达式，于本轮在线读取；用于 NULL、子查询基数及聚合等边界，不代表教师规定了 PostgreSQL 18 版本。
- 本版为 2026-10-20 的 LaTeX 重排；示例证据状态沿用原说明，未新增数据库实测。
- 输入 SHA-256：SQLpart1 `aa28c989fb3674e780418c78c483bba20ff5a50b7da067b0341e60410a68c8ed`；SQLpart2 `acd25cbd4bbd246f8901ada4c437c3f2d00873385961475cd4c941e9089bf45c`；Tutorial 2 `72c016526c1c0ff02c10fcb5f7f964c1184999c68422867102580093 6a6cabac`；syllabus 教学计划 `31380a7fe6504e46541b6db0bb2f24859c69188bfeb9681dca73354ebeb68295`。